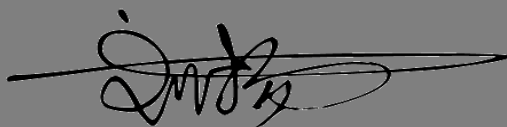


2025

操作系统实验指导

河南大学计算机与信息工程学院



Liu Yang
College of Computer Science and Information Engineering
Henan University
2025-9-10



目录

实验 1 LINUX 基本操作	1
一、实验目的	1
二、实验题目	1
三、背景材料	1
(一) 实验环境说明	1
(二) Virtual PC 使用	1
(三) Linux 常用命令	5
四、实验内容	7
五、实验报告书写要求	8
实验 2 LINUX 编程基础	9
一、实验目的	9
二、实验题目	9
三、背景材料	9
(一) 编辑器 vi 的使用	9
(二) gcc 和 make 的使用	11
(三) 需要用到的系统调用	13
四、实验内容	13
五、实验报告书写要求	14
实验 3 进程管理	15
实验目的	15
实验预备内容	15
实验要求	15
实验内容	16
1. 进程的创建	16
2. 进程的控制	16
3. 软中断通信	18
4. 进程的管道通信	22
实验 4 进程通信	25
实验目的	25
实验内容	25
1. 消息的创建, 发送和接收	25
2. 共享存储区的创建, 附接和断接	26
实验 5 任务调度模拟	29
实验目的	29
实验要求	29
背景材料	29

(一) 设计一个按优先数调度算法实现处理器调度的程序.....	29
(二) 设计一个按时间片轮转法实现处理器调度的程序。.....	30
四、实验内容.....	31
五、实验报告书写要求.....	31
实验 6 存储管理模拟.....	32
实验目的.....	32
实验要求.....	32
实验内容.....	33
实验 7 文件系统与 SHELL.....	43
实验目的.....	43
实验要求.....	43
实验提示.....	43
实验内容.....	43
附录 LINUX 系统调用函数说明、参数值及定义.....	49
1、FORK()	49
2、LOCKF(FILES,FUNCTION,SIZE):	49
3、MSGGET(KEY,FLAG):	49
4、MSGSEND(ID,MSGP,SIZE,FLAG):.....	49
5、MSGRCV(ID,MSGP,SIZE,TYPE,FLAG):	50
6、MSGCTL(ID,CMD,BUF):	50
7、SHMGET(KEY,SIZE,FLAG):.....	51
8、SHMAT(ID,ADDR,FLAG):	52
9、SHMDT(ADDR):	52
10、SHMCTL(ID,CMD,BUF):	52
11、SIGNAL(SIG,FUNCTION):.....	53

实验 1 Linux 基本操作

一、实验目的

熟悉 Linux 常用命令，为后续的 Linux 下编程实验打下基础。

二、实验题目

必做内容：进行基本文件和目录操作；修改口令；使用 `man` 获得帮助；查看进程状态；查看文件系统加载情况。

选做内容：用户管理，Shell 编程。

三、背景材料

（一）实验环境说明

因机房条件限制，用于操作系统实验的计算机配置不同，因此以如下两种不同的方式提供 Linux 操作系统环境：

- 1、使用 `telnet` 命令远程登录 Linux 服务器，默认信息如下（如有调整，以指导教师提供信息为准）：

服务器 ip 地址：172.20.1.55；

用户名：s+学号，例如学号为 070272001 的同学用户名为：s070272001；

口令：abc123。

- 2、使用虚拟机安装 Linux 系统：使用 Virtual PC，具体使用方法参见下一部分。

（二）Virtual PC 使用

Virtual PC 对内存有最低要求，一般主机低于 512M 内存、虚拟机内存低于 256M 则可用性很差；不推荐使用。由于虚拟机内存较小，不能安装较新版本的 Linux 发布，实验室提供的是 Redhat 9.0。试验过程中需要安装 Virtual PC(使用 **virtualpc2007setup.exe**)，然后利用虚拟硬盘文件(1.vhd)创建虚拟机。

【注】：上述两个文件的具体存放位置指导教师会在上机时给出说明；请所有学生注意纪律，不要删除或移动这两个文件。

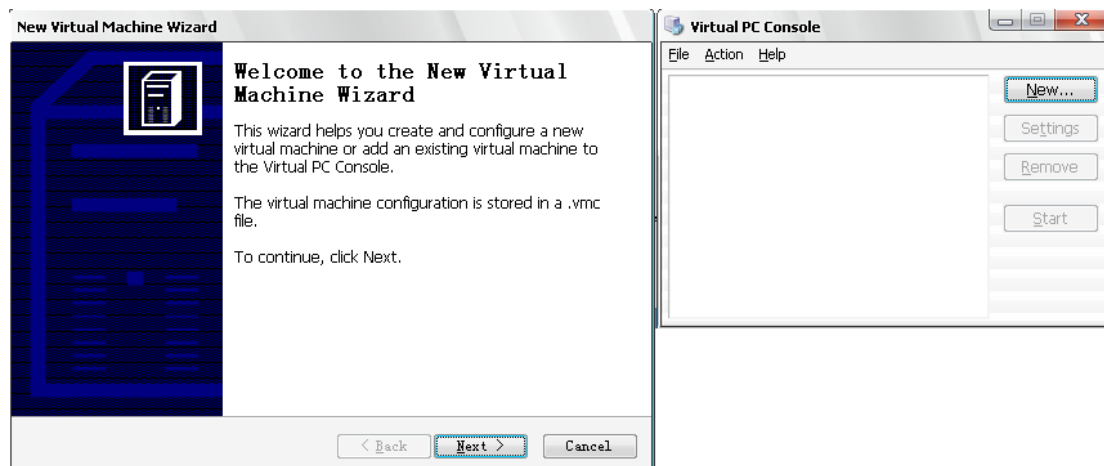
- 1、安装 virtual pc2007

双击 `virtualpc2007setup.exe` 启动安装过程，按照屏幕提示，完成软件安装。

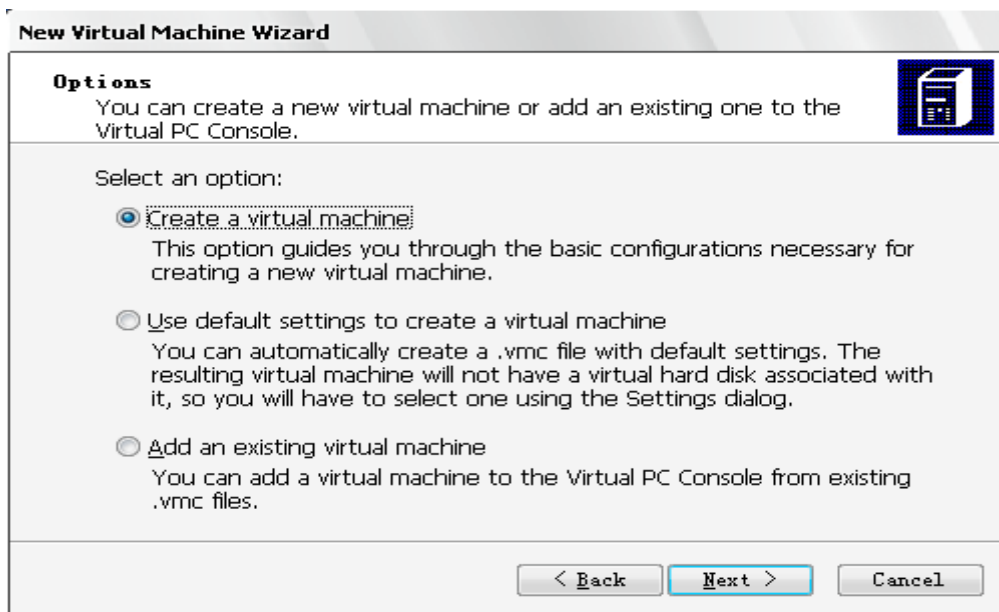
- 2、在 virtual pc2007 中安装 redhat linux 9

(1) 打开 virtual pc2007；

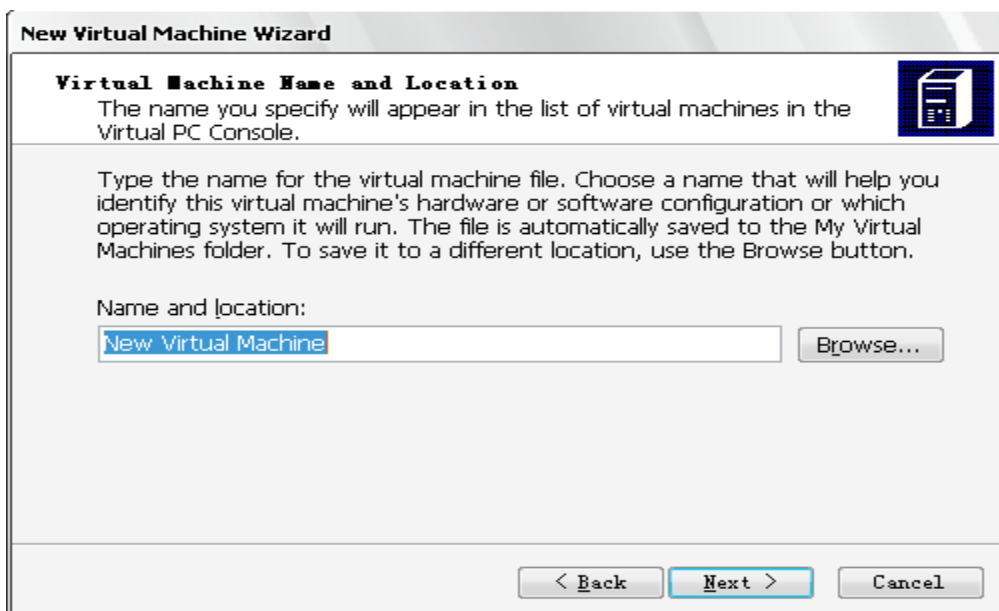
(2) 在 Wizard 对话框中点击“New...”，再点击“Next”；



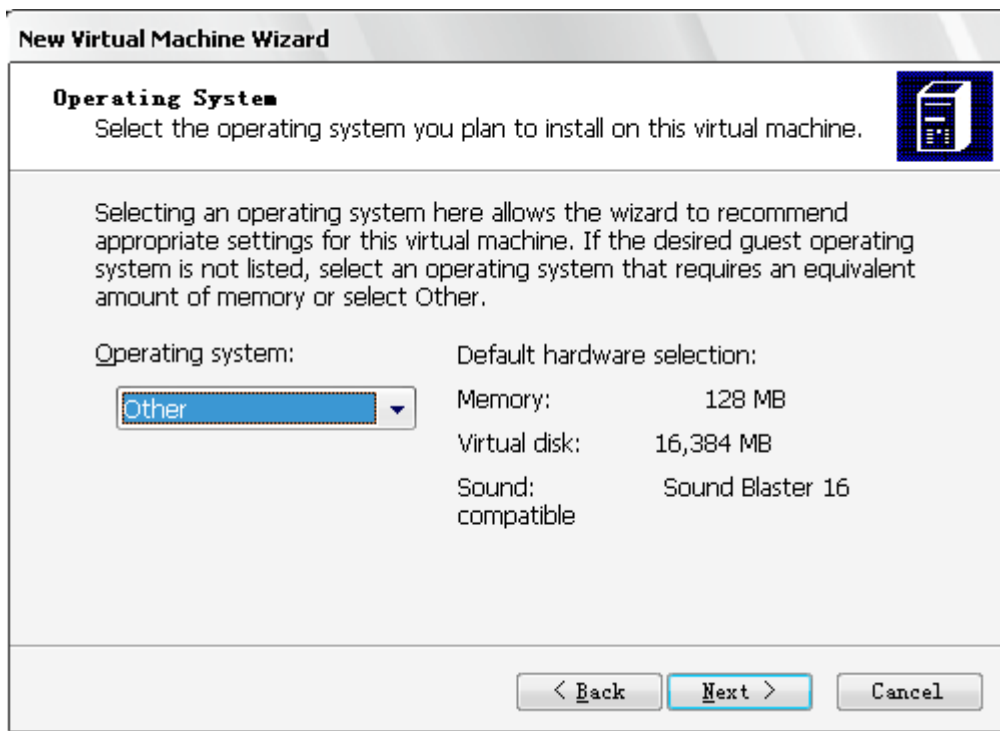
(3) 选择“Create a virtual machine”→“Next”；



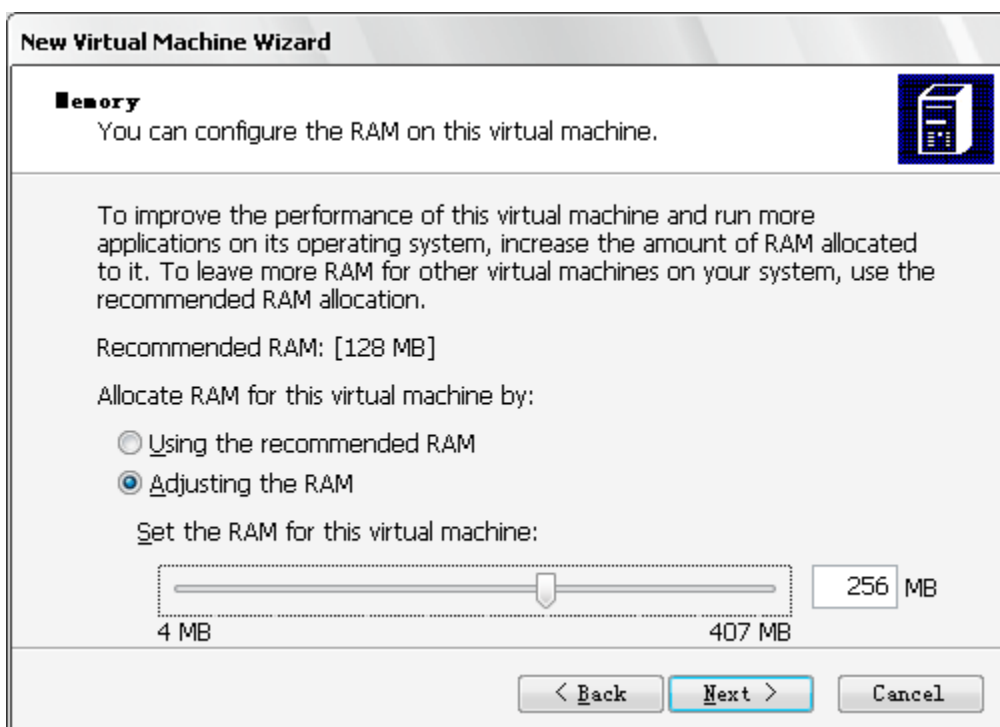
(4) 新建一个虚拟机，名字可以随意取，默认位置随意；



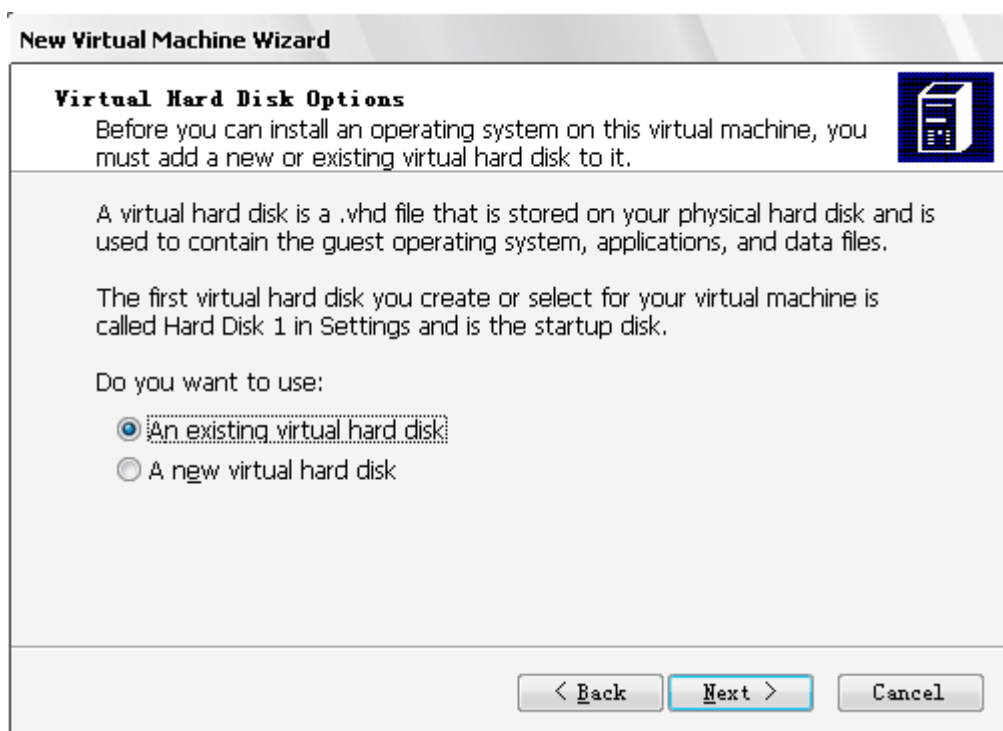
(5) 在操作系统类型选择时，由于 linux 不在列表之中，选择 “Other”；



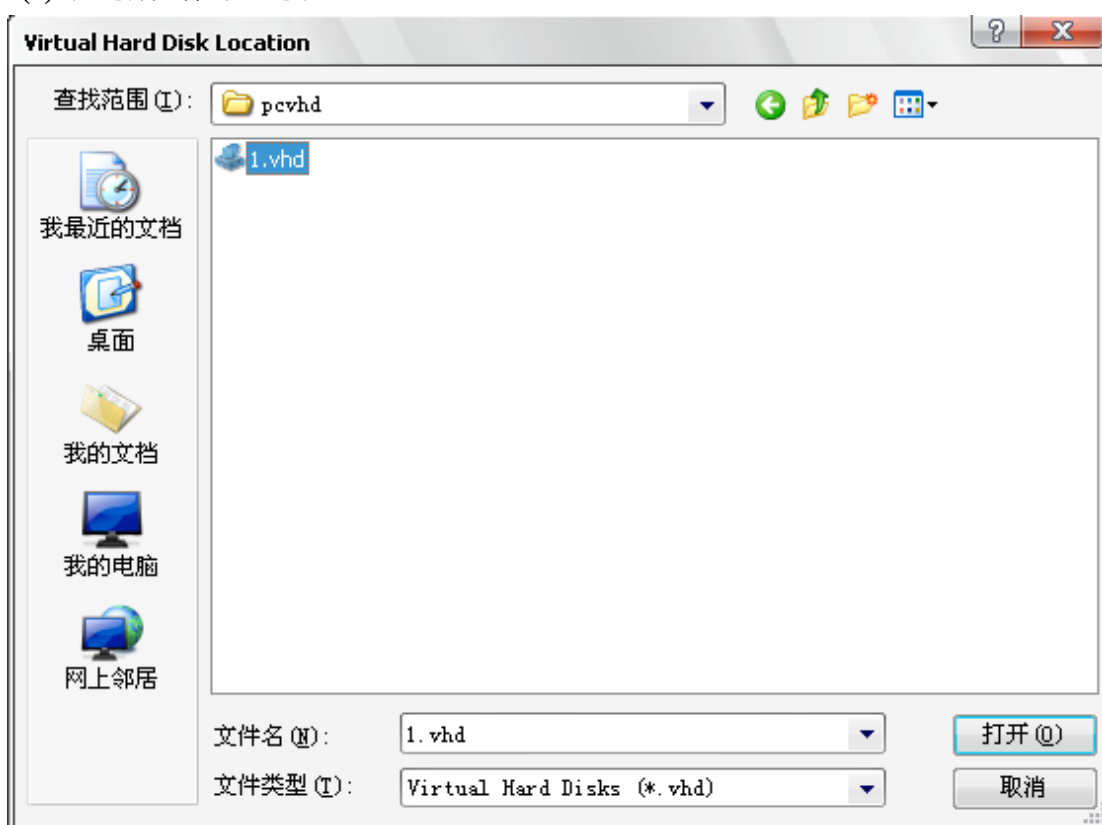
(6) 为该虚拟机分配内存，建议选择“**Adjusting the RAM**”，并设为 256MB；



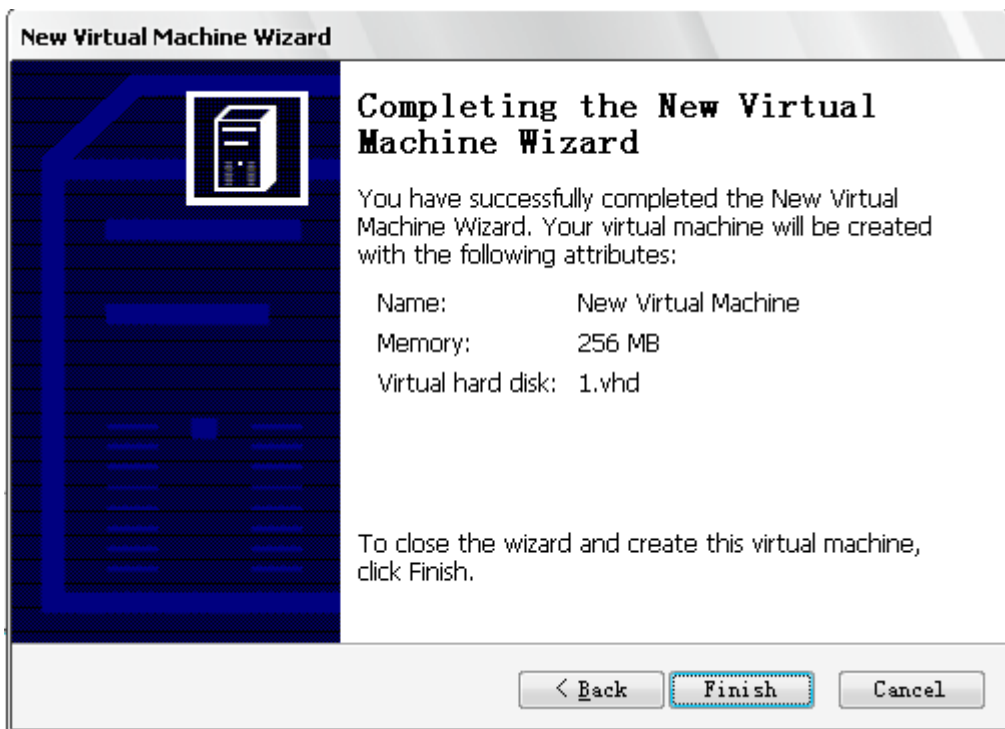
(7) 在设置虚拟硬盘时，选择使用已有的虚拟磁盘；



(8) 在选择文件时，选中 1.vhd;

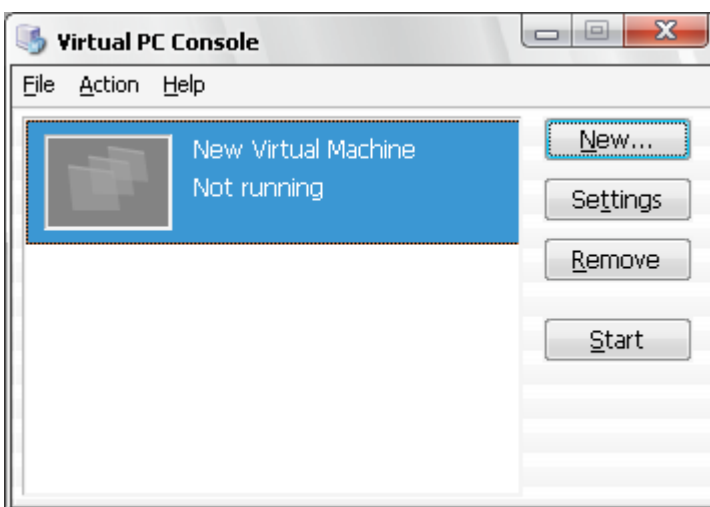


(9) 点击“Finish”完成配置。



3、打开 Redhat linux 9

(1) 打开虚拟机会看到如下的界面，点击“Start”。



(2) 经过一段时间的等待，会出现一个图形登录界面，使用用户名“root”、密码“123456”登录进入图形用户界面。提示：当鼠标在虚拟机界面中点击时，会发现光标已陷入其中，不能移动出虚拟机窗口，此时按住右边的 Alt 拖动鼠标即可将它释放出来。

（三）Linux 常用命令

1、有关目录的命令

(1) pwd 命令

pwd (即 print working directory, 打印工作路径) 命令的功能是显示当前的工作路径。如现在是在“/home/CAI”目录下，则可以用此命令来证实。例如：

```
$ pwd // “$” 表示在 Linux 系统的提示符下
/home/CAI // pwd 命令证明的确是在 “/home/CAI” 下
```

(2) cd 命令

cd 命令用来切换目录，假设在刚才的目录切换到另一目录，如下所示：

```
$ cd /usr/bin           // 将当前的目录转到 “/usr/bin” 下
$ pwd
/usr/bin
$ cd                     // 此命令返回工作目录
$ pwd
$ /home/CAI
```

(3) mkdir 命令

语法: mkdir 目录名

mkdir 命令用于建立新目录, 例如:

```
$ mkdir dir1           // 建立新目录 dir1
```

(4) rmdir / rm 命令

语法: rmdir 目录名 或 rm 目录名

该命令用于删除目录, 例如:

```
$ rmdir dir1           // 删除目录 dir1, 但 dir1 目录下必需没有文件存在, 否则无法删除
$ rm -r dir1           // 删除目录 dir1 及其子目录下所有文件, 注意-r 参数
```

2、有关文件的命令

(1) ls 命令

语法: ls [-atFlGR] [name]

ls 命令的功能是显示指定目录的内容, 例如:

```
$ cd
$ ls -a                //此命令显示当前工作目录下的所有文件 (参数
                        “a” 表示所有文件, “-” 号是用来控制参数)
```

显示时, 文件名前带 “.” 号表示隐含文件。

各参数代表的含义如下所示:

ls 列出当前目录下的文件名

ls -a 列出以.开始的隐藏文件的所有文件名

ls -t 依照文件最后修改时间的顺序列出文件名

ls -F 列出当前目录下的文件名及其类型, 以/结尾表示为目录名, 以*结尾表示为可执行文件, 以@结尾表示为符号链接。

ls -lg 列出目录下所有文件的权限、所有者、文件大小、修改时间及名称。

ls -l 同上, 并列出文件的所有者工作组名。

ls -R 显示出目录下以及其所有子目录的文件名 (包括了隐藏文件)

(2) cat 命令

cat 命令的功能是显示文件内容, 也可用于文件的连接。此命令常用来快速浏览文件, 使用方法如:

```
$ cat.bashrc
```

浏览文件的其他命令还有 more 等。

(3) cp 命令

语法: cp [options] 源文件 目标文件

cp 命令的功能是复制文件或目录, 可一次复制多个文件, 使用的参数如下:

-f: 强行覆盖已存在的目标文件。

-i: 在强行覆盖已存在的目标文件时给出提示。

-R: 整个目录复制。

(4) rm 命令

语法: rm 文件名

rm 命令用于删除文件。例如:

```
$ rm file1           // 删除文件名为 file1 的文件
$ rm file?           // 删除文件名中有五个字符且前四个字符为 file 的文件
$ rm f*              // 删除文件名中以 f 为字首的所有文件
```

(5) file 命令

file 命令的功能是探测文件和目录的类型。

(6) find 命令

语法: find [路径] [匹配表达式]

find 命令的功能是搜索目录中的文件

3、其他的命令

(1) man 命令

man 命令是联机帮助，类似于 DOS 中的 help，例如：

```
$ man ls
```

(2) ps 命令

ps 命令可以列出当前运行的进程相关信息。

(3) kill 命令

kill 命令的功能是中止一个过程，用法是：

```
kill [-s 信号] [p] [-a] 进程号
```

```
kill -l [信号]
```

(4) mount、umount 命令

mount、umount 命令的功能分别是安装与卸下一个文件系统。用法如下：

```
$ mount /mnt/floppy // 此命令将 A 驱动器安装上
```

```
$ umount /mnt/floppy // 此命令将 A 驱动器卸下
```

(5) passwd 命令

passwd 命令用于更改登录密码。普通用户只能更改自身密码，root 可以更改其它用户的密码。

(6) reboot 命令

语法：reboot

reboot 命令的功能是重新启动系统。

(7) su 命令

su 命令的功能是使普通用户以 root 帐号登录，用法是：键入 su 命令，Shell 要求 root 密码。键入密码按回车键则进入 root 帐号。

(8) date 命令

语法：date

date 命令的功能是打印或设置系统日期和时间，其用法与 DOS 中的 date 命令相似。

(9) clear 命令

clear 命令的功能是清除屏幕，用法如同 DOS 中的 cls 命令：

```
$ clear
```

四、实验内容

1、必做内容

(1) 修改口令

使用 telnet 远程进入 Linux 命令界面，修改当前用户的口令：

【注】 修改口令后，可以在 Linux 服务器上拥有一块存储空间（限额为 30M），通过 ftp 可以上传文件，使用如下形式的 url：<ftp://用户名@172.20.1.55>，根据提示输入口令。

(2) 基本目录和文件操作

使用 telnet 或者虚拟机方式进入 Linux 命令界面，完成基本的目录和文件操作如下：

查看登录进入后的主目录位置；

查看/目录下的目录结构；

在主目录下建立、删除、移动(重命名)子目录，形成树形结构；

在主目录下的子目录中复制、删除、移动(重命名)文件。

(3) 使用 man 命令获得帮助

使用 man 命令获得一些命令的详细信息，例如 man 自身、ps 命令、kill 命令等；

(4) 进程控制

使用 ps 命令查看当前进程状态，使用 kill 命令终止某个进程(例如当前使用的命令解释器进程)查看效果；

(5) 查看文件系统加载状况

使用 `mount` 命令查看当前文件系统加载情况；

(6) 其它

使用自己感兴趣的其它命令，并做分析和记录。

2、选做内容

(1) 用户管理

如果使用虚拟机进入 Linux 环境，可以用 `root` 身份进行用户管理，可以在命令界面下进行下列操作：

使用 `adduser` 命令创建用户；

使用 `passwd` 命令改变其它用户的口令；

对用户管理操作的效果进行验证；

使用 `userdel` 命令删除用户。

(2) 使用 GUI

如果使用虚拟机进入 Linux 环境，可以在图形用户界面下进行操作。尝试进行常规的目录和文件操作、网页浏览操作、系统设置操作。

(3) Shell 编程

根据个人兴趣，查找相关资料，进行 shell 编程实验。编写脚本文件应使用 shell 变量、循环控制结构等功能，例如编写一个遍历子目录进行某种操作的脚本。

五、实验报告书写要求

应在实验报告中说明如下事项：

- 1、进入 Linux 方式；
- 2、修改口令操作及验证方法；
- 3、登录进入系统后的主目录位置，进行文件及目录管理操作的记录和结果；
- 4、进行进程管理的操作情况和结果分析；
- 5、文件系统加载情况分析；
- 6、其它及选做内容：如果使用了其它命令，或者进行选做内容的实验，应在实验报告中记录操作过程并做相应分析。

实验 2 Linux 编程基础

一、实验目的

熟悉 Linux 下使用 C 语言编程的方法，通过在编程中使用进程控制系统调用功能，直观理解 Linux 下进程并发执行情况。

二、实验题目

使用 vi 编辑 C 语言源程序；分析例子程序并通过 man 命令熟悉相关系统调用，编写进程并发执行的程序；使用 gcc、make 等工具进行编译；执行编译生成的执行文件并分析结果。

三、背景材料

因实验用的 Linux 环境以字符界面为主，而 Linux 的字符界面并没有广泛使用的集成开发环境，经典的 C 语言开发模式还是使用文本编辑软件编写源程序，然后使用 gcc 和 make 工具进行编译链接，生成可执行文件。在编辑源程序环节，可以使用不同的文本编辑工具，例如在图形用户界面下可以使用 gedit 或者 kedit；字符界面下可以使用 vi、emacs、nano 等编辑器；甚至可以在其它环境下编辑，然后通过 ftp 上传。

（一）编辑器 vi 的使用

vi 或 vim 是 Linux 系统上经典的文本编辑工具，在字符界面下，广泛用于系统管理、程序开发。本实验要求熟悉 vi 基本操作，可以使用 vi 进行文本编辑。

1、调用 vi

最常用的命令模式是：vi filename

2、vi 的三种模式

Command（命令）模式，用于输入命令；

Insert（插入）模式，用于插入文本；

Visual（可视）模式，用于可视化的高亮并选定正文。

3、文件的保存和退出

Command 模式是 vi 的默认模式，如果处于其它模式时，要通过 ESC 键切换过来，接着再输入:号时，vi 会在屏幕的最下方等待输入命令；

```
:w 保存；  
:w filename 另存为 filename；  
:wq! 保存退出；  
:wq! filename 注：以 filename 为文件名保存后退出；  
:q! 不保存退出；  
:x 应该是保存并退出，功能和:wq!相同
```

4、光标移动；

按 ESC 进入 Command 模式后，可以用下面的一些键位来移动光标；

j 向下移动一行;
k 向上移动一行;
h 向左移动一个字符;
l 向右移动一个字符;
ctrl+b 向上移动一屏;
ctrl+f 向下移动一屏;
向上箭头 向上移动;
向下箭头 向下移动;
向左箭头 向左移动;
向右箭头 向右移动;

对于 j、k、l 和 h 键，还能在这些命令的前面加上数字，比如 3j，表示向下移动 3 行。

5、进入插入模式

i 在光标之前插入;
a 在光标之后插入;
I 在光标所在行的行首插入;
A 在光标所在行的行末插入;
o 在光标所在的行的上面插入一行;
O 在光标所在的行的下面插入一行;
s 删除光标后的一个字符，然后进入插入模式;
S 删除光标所在的行，然后进入插入模式;

6、文本内容的删除

x 删除一个字符;
#x 删除几个字符，#表示数字，比如 3x;
dw 删除一个单词;
#dw 删除几个单词，#用数字表示，比如 3dw 表示删除三个单词;
dd 删除一行;
#dd 删除多个行，#代表数字，比如 3dd 表示删除光标行及光标的下两行;
d\$ 删除光标到行尾的内容;
J 清除光标所处的行与上一行之间的空格，把光标行和上一行接在一起;

7、可视模式

在最新的 Linux 发行版本中，vi 提供了可视模式。按 ESC 键，然后按 v 就进入可视模式；退出可视模式，还是用 ESC 键。可视模式提供了友好的选取文本范围，以高亮显示。

8、复制和粘帖的操作

复制：对于可视模式下选中的文本，可以按 y 进行复制；按 yy 可以复制当前光标所在行。此外，删除也带有剪切的意思，当我们删除文字时，相应内容会自动复制；

粘帖：可以把光标移动到某处，然后按 p 或 shift+p 键进行粘帖，其中，p 在光标之后粘帖；shift+p 在光标之前粘帖。

9、行号

可以在命令模式输入 :set number 来打开行号显示，输入 :set number 来关闭行号显示。

10、查找功能

在命令模式下，输入 /或? 就可以进行查找：

/SEARCH 注：正向查找，按 n 键把光标移动到下一个符合条件的地方；
?SEARCH 注：反向查找，按 shift+n 键，把光标移动到下一个符合条件的

11、替换功能

按 ESC 键进入命令模式；

```
:s /SEARCH/REPLACE/g 注：把当前光标所处的行中的 SEARCH 单词，替换成 REPLACE，并把所有 SEARCH 高亮显示；  
:%s /SEARCH/REPLACE 注：把文档中所有 SEARCH 替换成 REPLACE；  
:#,# s /SEARCH/REPLACE/g 注：# 号表示数字，表示从多少行到多少行，把 SEARCH 替换成 REPLACE；
```

注：命令中 g 表示全局查找，在指定的替换范围之外，也会把 SEARCH 高亮显示。

（二）gcc 和 make 的使用

Linux 下编辑好的 C 语言程序，需要使用 gcc 编译工具(Linux 系统上一般也可以通过 cc 别名调用)进行编译连接，生成可执行文件。对于复杂的应用，可能把多个源程序分别编译，最后连接形成执行文件；为了有效管理这类项目，make 代码维护工具可以用来定义一系列规则来管理编译连接过程。

【注】Linux 下的可执行文件没有特别的后缀名约定，而是通过设置“可执行”属性来区分；另外，Linux 默认的执行文件搜索路径也不包括当前目录，所以编译生成的执行文件需要显示指明文件路径来执行。

1、gcc 的使用

(1) 简单使用方法

假设一个 C 语言源程序名为 hello.c，根据该文件生成执行文件的最简单命令为：

```
gcc hello.c
```

该命令将生成默认名字为 a.out 的执行文件；如果希望生成执行文件的名字不同，可以在命令中通过参数指定：

```
gcc -o hello hello.c
```

该命令将生成名字为 hello 的执行文件。

(2) 分解使用

实际上，gcc 所做的工作包括预编译、编译、连接等多个环节，上述简单用法隐含了对这几个环节的调用，如果需要分开调用，可以通过不同的命令行参数指定：

```
gcc -E hello.c -o hello.i    -E 参数指定进行预编译，预处理的宏定义插入到 hello.i 中；
```

```
gcc -c hello.i -o hello.o    -c 参数指定编译为目标代码 hello.o，也可以通过源文件直接生成(gcc -c hello.c)；
```

```
gcc hello.o -o hello        将生成可执行文件，如果由多个源程序文件生成，一般需要在生成时指定多个目标代码模块，例如：gcc hello.o e1.o s2.o -o hello
```

(3) 常用参数

gcc 命令行的常用参数如下：

- c 通知 GCC 取消链接步骤，即编译源码并在最后生成目标文件；
- Dmacro 定义指定的宏，使它能够通过源码中的#ifdef 进行检验；
- E 不经过编译预处理程序的输出而输送至标准输出；
- g3 获得有关调试程序的详细信息，它不能与-o 选项联合使用；
- Idirectory 在包含文件搜索路径的起点处添加指定目录；
- llibrary 提示链接程序在创建最终可执行文件时包含指定的库；
- O、-O2、-O3 将优化状态打开，该选项不能与-g 选项联合使用；
- S 要求编译程序生成来自源代码的汇编程序输出；
- v 启动所有警报；
- Wall 在发生警报时取消编译操作，即将警报看作是错误；
- Werror 在发生警报时取消编译操作，即把报警当作是错误；
- w 禁止所有的报警。

2、make 的使用

(1) make 工具简介

make 是一种代码维护工具，在大中型项目中，根据程序各个模块的更新情况，自动的维护和生成目标。能很好的做到“不多、不重、不漏”：

- 不多：只更新那些需要更新的文件，而不动那些并不过时的东西；
- 不重：是指当 make 对某个文件进行更新时，即使有很多文件过时，也将只更新一次；
- 不漏：是指他不会漏下任何应该更新的文件。

make 在使用时有一系列的规则，将根据这些规则来解释其配置脚本，以达到设计目的。配置脚本的缺省名是 makefile 或 Makefile，也可在命令行指定其它文件名。使用形式为：

```
make [option] [macrodef] [target]
```

option 指出 make 的工作行为，具体参数可参考 man 手册页。

(2) makefile 文件编写

make 工具使用的关键是编写高效、简洁、正确的 makefile，这需要学习和实践。虽然 make 工具的使用有较高的门槛，但是一旦掌握，其带来的灵活和便捷远远超过一般 IDE 所能提供的管理功能。

实际 makefile 文件编写涉及的知识很多，有专著专门进行讲解。此处只是介绍概貌，借助一个简单的例子来说明：假设一个项目 hello，依赖于 hello.o、e1.o、e2.o，而 hello.o 依赖于 hello.c、hello.h，e1.o 依赖于 e1.c、hello.h，e2.o 依赖于 e2.c；这样形成一棵依赖关系树，父节点依赖于子节点。使用 make 进行维护时，会对这棵树进行一次遍历，如果发现子节点形成的时间晚于父节点形成的时间便开始调用 makefile 中指定的命令进行维护。针对此例子的 makefile 示例如下：

```
hello: hello.o e1.o e2.o
    gcc -o hello hello.o e1.o e2.o
hello.o: hello.c hello.h
    gcc -c hello.c
e1.o: e1.c hello.h
    gcc -c e1.c hello.h
e2.o: e2.c
    gcc -c e2.c
```

makefile 放置在与项目源程序文件相同目录中，在该目录中执行 make hello 或者 make 命令(命令行未 target 时会默认使用 makefile 的第一个 target)，将会根据 makefile 中定义的依赖关系依次执行编译、连接各个命令，生成 hello 执行文件。

实际项目的 makefile 因为会使用大量的宏、条件变量以及缺省规则，对初学者而言较难理解。对于一般的开源项目，在编写 makefile 时，target 的命名都会遵循一些预定俗称的规则，常见的用法包括：

```
make all      编译项目中所有实际目标
make install  进行安装
make clean    清除所有中间文件
```

(3) 缺省规则和简单用法

make 的缺省规则是系统或者自定义的一些规则，在定义时不指出全名，而是根据文件扩展名定义一类依赖关系和相关命令。例如，与 c 语言相关的缺省规则包括：

```
.c:
    $(CC) $(FLAGS) -o $@ $<
.c.o:
    $(CC) $(FLAGS) -c $@ $<
```

两条规则的含义为：对于.c 扩展名的文件，调用编译器(通过环境变量 CC 定义，默认为 gcc)生成与.c 文件对应名字的执行文件；对于.o、.c 扩展名配对的情况，调用编译器进行编译，生成与.c 文件对应名字的.o 目标文件。

根据这些缺省规则，可以在简单情况下不编写 makefile 使用 make 工具。例如，对于根据 hello.c 生成 hello 执行文件的情况，可以直接使用命令

```
make hello
```

效果等同于：gcc -o hello hello.c

（三）需要用到的系统调用

实验要求在例子程序基础上编程验证 Linux 下进程并发情况，需要用到的系统调用和库函数在下面列出，详细的使用方法说明通过“man 2 系统调用名”或者“man 3 函数名”命令获取。

fork()	创建一个子进程，通过返回值区分是在父进程还是子进程中执行；
wait()	等待子进程执行完成；
sleep()	导致调用进程睡眠指定秒数；
getpid()	获取当前进程 id；
getppid()	获取父进程 id。

四、实验内容

为便于掌握 Linux 环境下进行 C 语言编程的基本技能，本实验提供源程序例子，实验内容要在给出的例子程序基础上，根据要求进行修改，对执行结果进行分析。

1、子进程对存储空间的复制

(1) 使用文本编辑器输入源程序

输入如下源程序：

```
#include <sys/types.h>
#include <unistd.h>
#include <stdio.h>

int main(void)
{
    int x, i;

    printf("Input a initial value for i: ");
    scanf("%d", &i);
    while((x=fork())!=-1);
    if(x==0)        /* child run */
    {
        printf("When child runs, i=%d\n", i);
        printf("Input a value in child: ");
        scanf("%d", &i);
        printf("i=%d\n", i);
    }
    else            /* parent run */
    {
        wait();
        printf("After child runs, in parent, i=%d\n", i);
    }
}
```

(2) 预测程序的执行结果

阅读程序，根据自己的理解，预期程序执行后的结果。

(3) 实际执行结果分析

编译生成执行文件，执行后记录结果，说明与预期的结果是否一致。分析为什么是这样的结果。

2、父子进程执行过程分析

(1) 按照给定框架编程

按照如下程序框架，完成源程序编写：

```
// ①
pid=fork();
// ②
if(pid==0)
{
    sleep(3);
    printf("Child: pid=%d, ppid=%d\n", getpid(), getppid());
}
else
{
    printf("Parent: Child=%d, pid=%d, ppid=%d\n", pid, getpid(), getppid());
    wait();
    printf("After Child ends.\n");
}
printf("In which process?\n"); // ③
```

(2) 预测程序的执行结果

阅读程序，根据自己的理解，预期程序执行后的结果。

(3) 实际执行结果分析

编译生成执行文件，执行后记录结果，说明与预期的结果是否一致。分析为什么是这样的结果。

(4) 修改程序并分析执行结果

把程序框架中最后一句输出语句（位置③处）分别移至位置①和②处，预期输出结果是什么？实际执行结果如何？分析原因。

(5) 修改程序验证父子进程关系

修改程序，使父进程先执行完成，验证子进程是否会一起终止？如果不是，前后子进程的父进程号是否变化？记录并分析结果。

五、实验报告书写要求

应在实验报告中说明如下事项：

- 1、所使用的文本编辑器；
- 2、编译生成执行文件的命令；
- 3、相关程序的名称及存储位置，以便指导教师抽查；
- 4、实验内容 1 的结果：包括预期结果和实际执行结果，以及结果分析；
- 5、实验内容 2 的结果：包括预期结果和实际执行结果，对结果的分析；按照要求进行修改后的预期结果、实际结果及分析。

实验 3 进程管理

实验目的

- (1) 加深对进程概念的理解，明确进程和程序的区别。
- (2) 进一步认识并发执行的实质。
- (3) 分析进程竞争资源现象，学习解决进程互斥的方法。
- (4) 了解 Linux 系统中进程通信的基本原理。

实验预备内容

- (1) 阅读 Linux 的 sched.h 源文件，加深对进程管理概念的理解。
- (2) 阅读 Linux 的 fork.c 源文件，分析进程的创建过程。

实验要求

(1) 进程的创建

编写一段源程序，使系统调用 `fork()` 创建两个子进程，当此程序运行时，在系统中有一个父进程和两个子进程活动。让每一个进程在屏幕上显示一个字符：父进程显示字符“a”；子进程分别显示字符“b”和字符“c”。试观察纪录屏幕上的显示结果，并分析原因。

(2) 进程的控制

修改已编写的程序，将每个进程输出一个字符改为每个进程输出一句话，在观察程序执行时屏幕出现的现象，并分析原因。

如果在程序中使用调用 `lockf()` 来给每一个子进程加锁，可以实现进程之间的互斥，观察并分析出现的现象。

(3) ①编写一段程序，使其实现进程的软中断通信。

要求：使用系统调用 `fork()` 创建两个子进程，再用系统调用 `signal()` 让父进程捕捉键盘上来的中断信号（即按 DEL 键）；当捕捉到中断信号后，父进程用系统调用 `Kill()` 向两个子进程发出信号，子进程捕捉到信号后分别输出下列信息后终止：

Child Process1 is Killed by Parent!

Child Process2 is Killed by Parent!

父进程等待两个子进程终止后，输出如下的信息后终止

Parent Process is Killed!

②在上面的程序中增加语句 `signal (SIGNAL, SIG-IGN)` 和 `signal (SIGQUIT, SIG-IGN)`，观察执行结果，并分析原因。

(4) 进程的管道通信

编制一段程序，实现进程的管理通信。

使用系统调用 `pipe()` 建立一条管道线；两个子进程 P1 和 P2 分别向管道中写一句话：

Child 1 is sending a message!

Child 2 is sending a message!

而父进程则从管道中读出来自于两个子进程的信息，显示在屏幕上。

要求父进程先接收子进程 P1 发来的消息，然后再接收子进程 P2 发来的消息。

思考

- (1) 系统是怎样创建进程的？
- (2) 可执行文件加载时进行了哪些处理？
- (3) 当首次调用新创建进程时，其入口在哪里？
- (4) 进程通信有什么特点？

实验内容

1. 进程的创建

〈任务〉

编写一段程序，使用系统调用 `fork()` 创建两个子进程。当此程序运行时，在系统中有一个父进程和两个子进程活动。让每一个进程在屏幕上显示一个字符；父进程显示字符“a”，子进程分别显示字符“b”和“c”。试观察记录屏幕上的显示结果，并分析原因。

〈程序〉

```
#include<stdio.h>
main()
{
int p1,p2;
    if(p1=fork())                /*子进程创建成功*/
        putchar('b');
    else
    {
        if(p2=fork())            /*子进程创建成功*/
            putchar('c');
        else
            putchar('a');        /*父进程执行*/
    }
}
```

<运行结果>

bca(有时会出现 abc 的任意的排列)

分析：从进程执行并发来看，输出 abc 的排列都是有可能的。

原因：fork()创建进程所需的时间虽然可能多于输出一个字符的时间，但各个进程的时间片的获得却不是一定是顺序的，所以输出 abc 的排列都是有可能的。

2. 进程的控制

<任务>

修改已编写好的程序，将每个程序的输出由单个字符改为一句话，再观察程序执行时屏幕上出现的现象，并分析其原因。如果在程序中使用系统调用 `lockf()`来给每个程序加锁，可以实现进程之间的互斥，观察并分析出现的现象。

〈程序 1〉

```
#include<stdio.h>
main()
{
    int p1,p2,i;
    if(p1=fork())
    {
        for(i=0;i<500;i++)
            printf("parent%d\n",i);
        wait(0); /* 保证在子进程终止前，父进程不会终止*/
        exit(0);
    }
}
```

```

}
else
{
    if(p2=fork())
    {
        for(i=0;i<500;i++)
            printf("son %d\n",i);
        wait(0); /* 保证在子进程终止前，父进程不会终止*/
        exit(0); /*向父进程信号 0 且该进程推出*/
    }
else
    {
        for(i=0;i<500;i++)
            printf("grandchild %d\n",i);
        exit(0);
    }
}
}
}

```

〈运行结果〉

```

parent....
son...
grandchild...
grandchild...
或 grandchild
...son
...grandchild
...son
...parent

```

分析：由于函数 `printf()` 输出的字符串之间不会被中断，因此，每个字符串内部的字符顺序输出时不变。但是，由于进程并发执行时的调度顺序和父子进程的抢占处理机问题，输出字符串的顺序和先后随着执行的不同而发生变化。这与打印单字符的结果相同。

〈程序 2〉

```

#include<stdio.h>
main()
{
    int p1,p2,i;
    if(p1=fork())
    {
        lockf(1,1,0);
        for(i=0;i<500;i++)
            printf("parent %d\n",i);
        lockf(1,0,0);
        wait(0); /* 保证在子进程终止前，父进程不会终止*/
        exit(0);
    }
}

```

```

}
    else
    {
        if(p2=fork())
        {
            lockf(1,1,0);
            for(i=0;i<500;i++)
printf("son %d\n",i);
            lockf(1,0,0);
            wait(0); /* 保证在子进程终止前，父进程不会终止*/
            exit(0);

        }
    else
    {
        lockf(1,1,0);
        for(i=0;i<500;i++)
printf("daughter %d\n",i);
        lockf(1,0,0);
        exit(0);
    }
}
}

```

<运行结果>

输出 parent 块,son 块,grandchild 块的顺序可能不同，但是每个块的输出过程不会被打断。

分析：因为上述程序执行时，lockf(1,1,0)锁定标准输出设备，lockf(1,0,0)解锁标准输出设备，在 lockf(1,1,0)与 lockf(1,0,0)中间的 for 循环输出不会被中断，加锁与不加锁效果不相同。

3. 软中断通信

〈任务 1〉

编制一段程序，使用系统调用 fork()创建两个子进程，再用系统调用 signal()让父进程捕捉键盘上来的中断信号（即按 ctrl+c 键），当捕捉到中断信号后，父进程用系统调用 kill()向两个子进程发出信号，子进程捕捉到信号后，分别输出下列信息后终止：

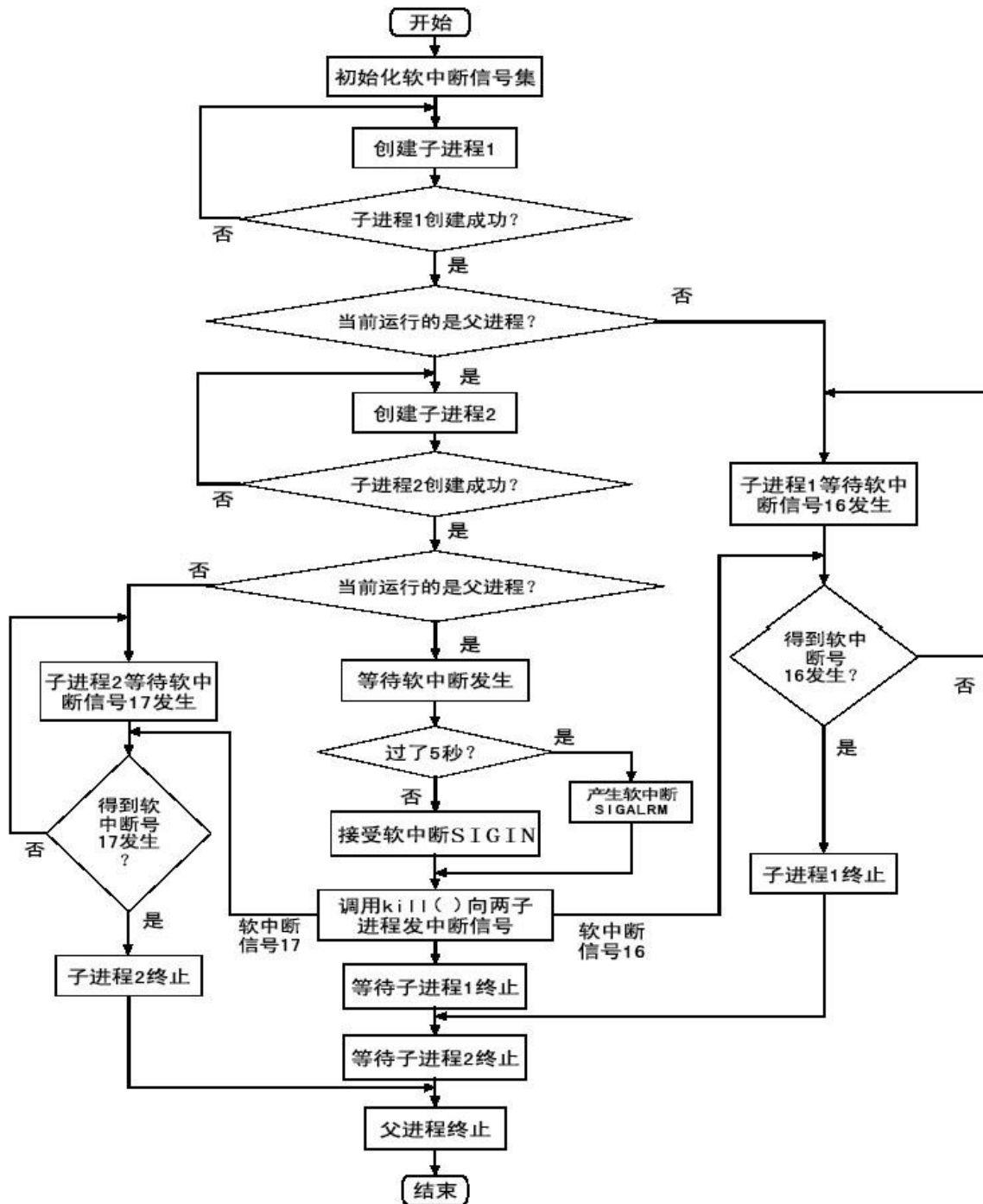
child process1 is killed by parent!

child process2 is killed by parent!

父进程等待两个子进程终止后，输出以下信息后终止：

parent process is killed!

<程序流程图>



〈程序〉

```
#include<stdio.h>
#include<signal.h>
#include<unistd.h>
```

```
void waiting(),stop(),alarming();
int wait_mark;
```

```

main()
{
    int p1,p2;
    if(p1=fork())          /*创建子进程 p1*/
    {
        if(p2=fork())      /*创建子进程 p2*/
        {
            wait_mark=1;
            signal(SIGINT,stop);    /*接收到^c 信号，转 stop*/
            signal(SIGALRM,alarming);/*接受 SIGALRM
            waiting();

            kill(p1,16);          /*向 p1 发软中断信号 16*/
            kill(p2,17);          /*向 p2 发软中断信号 17*/

            wait(0);              /*同步*/
            wait(0);
            printf("parent process is killed!\n");
            exit(0);
        }
        else
        {
            wait_mark=1;
            signal(17,stop);
            signal(SIGINT,SIG_IGN); /*忽略 ^c 信号*/
            while (wait_mark!=0);
            lockf(1,1,0);
            printf("child process2  is killed by parent!\n");
            lockf(1,0,0);
            exit(0);
        }
    }
    else
    {
        wait_mark=1;
        signal(16,stop);
        signal(SIGINT,SIG_IGN); /*忽略^c 信号*/
        while (wait_mark!=0)
        lockf(1,1,0);
        printf("child process1 is killed by parent!\n");
        lockf(1,0,0);
        exit(0);
    }
}

```

```

void waiting()
{
    sleep(5);
if (wait_mark!=0)
    kill(getpid(),SIGALRM);

}
void alarming()
{
    wait_mark=0;
}
void stop()
{
    wait_mark=0;
}

```

<运行结果>

不做任何操作等待五秒钟父进程回在子进程县推出后退出，并打印退出的顺序；或者点击 ctrl+C 后程序退出并打印退出的顺序。

〈任务 2〉

在上面的任务 1 中，增加语句 `signal(SIGINT,SIG_IGN)`和语句 `signal(SIGQUIT,SIG_IGN)`，观察执行结果，并分析原因。这里，`signal(SIGINT,SIG_IGN)`和 `signal(SIGQUIT,SIG_IGN)`分别为忽略键信号以及忽略中断信号。

<程序>

```

#include<stdio.h>
#include<signal.h>
#include<unistd.h>

int pid1,pid2;
int EndFlag=0;
int pf1=0;
int pf2=0;

void IntDelete()
{
    kill(pid1,16);
    kill(pid2,17);

}

void Int1()
{
    printf("child process 1 is killed !by parent\n");
    exit(0);
}

```

```

}
void Int2()
{
    printf("child process 2 is killed !by parent\n");
    exit(0);
}

main()
{
    int exitpid;
    if(pid1=fork())
    {
        if(pid2=fork())
        {
            signal(SIGINT,IntDelete);
            waitpid(-1,&exitpid,0);
            waitpid(-1,&exitpid,0);
            printf("parent process is killed\n");
            exit(0);
        }
        else
        {
            signal(SIGINT,SIG_IGN);
            signal(17,Int2);
            pause();
        }
    }
    else
    {
        signal(SIGINT,SIG_IGN);
        signal(16,Int1);
        pause();
    }
}

```

〈运行结果〉

请读者将上述程序输入计算机后，执行并观察。

4. 进程的管道通信

〈任务〉

编制一段程序，实现进程的管道通信。使用系统调用 `pipe()` 建立一条管道线。两个子进程 `p1` 和 `p2` 分别向通道个写一句话：

child1 process is sending message!

child2 process is sending message!

而父进程则从管道中读出来自两个进程的信息，显示在屏幕上。

〈程序〉

```
#include <unistd.h>
```

```

#include <signal.h>
#include <stdio.h>
int pid1,pid2;

main( )
{
int fd[2];
char outpipe[100],inpipe[100];
pipe(fd);          /*创建一个管道*/
while ((pid1=fork( ))!=-1);
if(pid1==0)
{
lockf(fd[1],1,0);
    sprintf(outpipe,"child 1 process is sending message!");
    /*把串放入数组 outpipe 中*/
    write(fd[1],outpipe,50);    /*向管道写长为 50 字节的串*/
    sleep(5);          /*自我阻塞 5 秒*/
    lockf(fd[1],0,0);
    exit(0);
}
else
{
while((pid2=fork( ))!=-1);
    if(pid2==0)
    {
lockf(fd[1],1,0);    /*互斥*/
        sprintf(outpipe,"child 2 process is sending message!");
        write(fd[1],outpipe,50);
        sleep(5);
        lockf(fd[1],0,0);
        exit(0);
    }
    else
    {
wait(0);            /*同步*/
        read(fd[0],inpipe,50);    /*从管道中读长为 50 字节的串*/
        printf("%s\n",inpipe);
        wait(0);
        read(fd[0],inpipe,50);
        printf("%s\n",inpipe);
        exit(0);
    }
}
}
}

```

〈运行结果〉

延迟 5 秒后显示:

child1 process is sending message!

再延迟 5 秒:

child2 process is sending message!

〈分析〉

请读者自行完成。

<思考>

1、程序中的 sleep(5)起什么作用？

2、子进程 1 和 2 为什么也能对管道进行操作？

实验 4 进程通信

实验目的

Linux 系统的进程通信机构 (IPC) 允许在任意进程间大批量地交换数据。本实验的目的是了解和熟悉 Linux 支持的消息通讯机制及信息量机制。

2、实验预备内容

阅读 Linux 系统的 msg.c、sem.c 和 shm.c 等源码文件，熟悉 Linux 的三种机制。

3、实验内容

(1) 消息的创建，发送和接收。

①使用系统调用 msgget (), msgsnd (), msgrev (), 及 msgctl () 编制一长度为 1k 的消息的发送和接收程序。

②观察上面的程序，说明控制消息队列系统调用 msgctl () 在此起什么作用？

(2) 共享存储区的创建、附接和段接。

使用系统调用 shmget(),shmat(),sgmdt(),shmctl(),编制一个与上述功能相同的程序。

比较上述 (1)，(2) 两种消息通信机制中数据传输的时间。

实验内容

1. 消息的创建，发送和接收

〈任务〉

使用系统调用 msgget (), msgsnd (), msgrev () 及 msgctl() 编制一长度为 1K 的消息发送和接收的程序。

〈程序设计〉

为了便于操作和观察结果，用一个程序为“引子”，先后 fork() 两个子进程，SERVER 和 CLIENT，进行通信。

SERVER 端建立一个 Key 为 75 的消息队列，等待其他进程发来的消息。当遇到类型为 1 的消息，则作为结束信号，取消该队列，并退出 SERVER。SERVER 每接收到一个消息后显示一句 “(server)received”。

CLIENT 端使用 Key 为 75 的消息队列，先后发送类型从 10 到 1 的消息，然后退出。最后的一个消息，既是 SERVER 端需要的结束信号。CLIENT 每发送一条消息后显示一句 “(client)sent”。

父进程在 SERVER 和 CLIENT 均退出后结束。

〈程序〉

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/msg.h>
#include <sys/ipc.h>
#define MSGKEY 75          /*定义关键词 MEGKEY*/
struct msgform             /*消息结构*/
{
    long mtype;
    char mtexe[100];        /*文本长度*/
}msg;
int msgqid,i;

void CLIENT()
{
    int i;
    msgqid=msgget(MSGKEY,0777|IPC_CREAT);
```

```

    for(i=10;i>=1;i--)
    {
        msg.mtype=i;
        printf("(client)sent\n");
        msgsnd(msgqid,&msg,1030,0);      /*发送消息 msg 入 msgid 消息队列*/
    }
    exit(0);
}

void SERVER( )
{
    msgqid=msgget(MSGKEY,0777|IPC_CREAT); /*由关键字获得消息队列*/
    do
    {
        msgrcv(msgqid,&msg,1030,0,0); /*从队列 msgid 接受消息 msg*/
        printf("(server)receive\n");
    }while(msg.mtype!=1);              /*消息类型为 1 时，释放队列*/
    msgctl(msgqid, IPC_RMID,0);

}

main()
{
    if(fork())
    {
        SERVER();
        wait(0);
    }
    else CLIENT( );
}

```

<结果>

从理想的结果来说，应当是每当 Client 发送一个消息后，server 接收该消息，Client 再发送下一条。也就是说“(Client)sent”和“(server)received”的字样应该在屏幕上交替出现。实际的结果大多是，先由 Client 发送两条消息，然后 Server 接收一条消息。此后 Client

Server 交替发送和接收消息.最后一次接收两条消息. Client 和 Server 分别发送和接收了 10 条消息,与预期设想一致

<分析>

message 的传送和控制并不保证完全同步,当一个程序不再激活状态的时候,它完全可能继续睡眠,造成上面现象,在多次 send message 后才 receive message.这一点有助于理解消息转送的实现机理.

2. 共享存储区的创建, 附接和断接

<任务>

使用系统调用 shmget(),sgmat(),smgdt(),shmctl()编制一个与上述功能相同的程序.

<程序设计>

- (1) 为了便于操作 和 观察结果，用一个 程序为“引子”，先后 fork() 两个子进程，SERVER 和 CLIENT，进行通信。
- (2) SERVER 端建立一个 KEY 为 75 的共享区,并将第一个字节置为-1.作为数据空的标志.等待其他进程发来的消息.当该字节的值发生变化时,表示收到了该消息,进行处理.然后再次把它 的值设为-1.如果遇到的值为 0,则视为结束信号,取消该队列,并退出 SERVER.SERVER 每接收到一次数据后显示”(server)received”.
- (3) CLIENT 端建立一个为 75 的共享区,当共享取得第一个字节为-1 时, Server 端空闲,可发送请求 . CLIENT 随即填入 9 到 0.期间等待 Server 端再次空闲.进行完这些操作后, CLIENT 退出. CLIENT 每发送一次数据后显示”(client)sent”.
- (4)父进程在 SERVER 和 CLIENT 均退出后结束.

<程序>

```
#include<sys/types.h>
#include<sys/msg.h>
#include<sys/ipc.h>
#define SHMKEY 75 /*定义共享区关键词*/
int shmid,i;
int *addr;

CLIENT()
{
    int i;
    shmid=shmget(SHMKEY,1024, 0777|IPC_CREAT); /*获取共享区，长度 1024，关键词 SHMKEY*/
    addr=shmat(shmid,0,0); /*共享区起始地址为 addr*/
    for(i=9;i>=0;i--)
    {
        while(*addr!= -1);
        printf("(client)sent\n"); /*打印（client）sent*/
        *addr=i; /*把 i 赋给 addr*/
    }
    exit(0);
}

SERVER()
{
    do
    {
        while(*addr == -1);
        printf("(server)received\n%d",*addr); /*服务进程使用共享区*/
        if(*addr!=0)
            *addr=-1;
    } while(*addr);
    wait(0);
    shmctl(shmid,IPC_RMID,0);
}
```

```

main()
{
    shmid=shmget(SHMKEY,1024,0777|IPC_CREAT); /*创建共享区*/
    addr=shmat(shmid,0,0); /*共享区起始地址为 addr*/
    *addr=-1;
    if(fork())
    {
        SERVER();
    }
    else
    {
        CLIENT();
    }
}

```

<结果>

运行的结果和预想的完全一样。但在运行的过程中，发现每当 client 发送一次数据后，server 要等大约 0.1 秒才有响应。同样，之后 client 又需要等待大约 0.1 秒才发送下一个数据。

<分析>

出现上述的应答延迟的现象是程序设计的问题。当 client 端发送了数据后，并没有任何措施通知 server 端数据已经发出，需要由 client 的查询才能感知。此时，client 端并没有放弃系统的控制权，仍然占用 CPU 的时间片。只有当系统进行调度时，切换到了 server 进程，再进行应答。这个问题，也同样存在于 server 端到 client 的应答过程之中。

3 比较两种消息通信机制中的数据传输的时间

由于两种机制实现的机理和用处都不一样，难以直接进行时间上的比较。如果比较其性能，应更加全面的分析。

消息队列的建立比共享区的设立消耗的资源少。前者只是一个软件上设定的问题，后者需要对硬件操作，实现内存的映像，当然控制起来比前者复杂。如果每次都重新进行队列或共享的建立，共享区的设立没有什么优势。

当消息队列和共享区建立好后，共享区的数据传输，受到了系统硬件的支持，不耗费多余的资源；而消息传递，由软件进行控制和实现，需要消耗一定的 CPU 资源。从这个意义上讲，共享区更适合频繁和大量的数据传输。消息的传递，自身就带有同步的控制。当等到消息的时候，进程进入睡眠状态，不再消耗 CPU 资源。而共享队列如果不借助其他机制进行同步，接受数据的一方必须进行不断的查询，白白浪费了大量的 CPU 资源。可见消息方式的使用更加灵活。

实验 5 任务调度模拟

实验目的

在采用多道程序设计的系统中，往往有若干个进程同时处于就绪状态。当就绪状态进程个数大于处理器数时，就必须依照某种策略来决定哪些进程优先占用处理器。本实验模拟在单处理器情况下处理器调度，帮助自己加深了解处理器调度的工作。

实验要求

选择一个调度算法，实现处理器调度。

背景材料

阅读教材《计算机操作系统》第二章和第三章，掌握进程管理及调度相关概念和原理。

(一) 设计一个按优先数调度算法实现处理器调度的程序

[提示]:

1、假定系统有五个进程，每一个进程用一个进程控制块 PCB 来代表，进程控制块的格式为:

进程名
指针
要求运行时间
优先数
状态

进程名——作为进程的标识，假设五个进程的进程名分别为 P₁, P₂, P₃, P₄, P₅。

指针——按优先数的大小把五个进程连成队列，用指针指出下一个进程的进程控制块的首地址，最后一个进程中的指针为“0”。

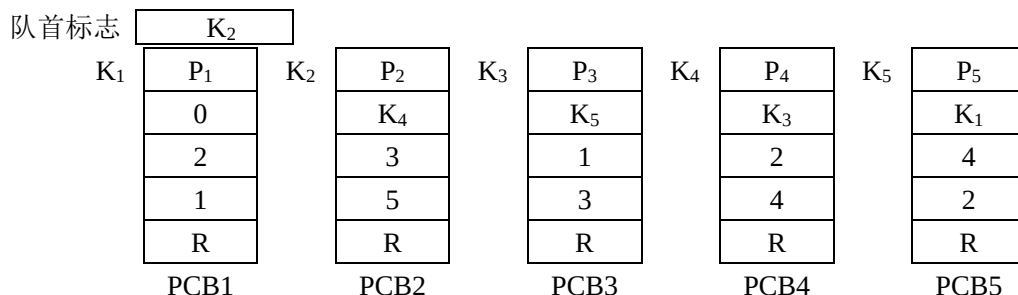
要求运行时间——假设进程需要运行的单位时间数。

优先数——赋予进程的优先数，调度时总是选取优先数大的进程先执行。

状态——可假设有两种状态，“就绪”状态和“结束”状态。五个进程的初始状态都为“就绪”，用“R”表示，当一个进程运行结束后，它的状态为“结束”，用“E”表示。

2、在每次运行你所设计的处理器调度程序之前，为每个进程任意确定它的“优先数”和“要求运行时间”。**本实验由于为了检查的方便，优先数和运行时间采用下表中的数值。**

3、为了调度方便，把五个进程按给定的优先数从大到小连成队列。用一单元指出队首进程，用指针指出队列的连接情况。例如:



4、处理器调度总是选队首进程运行。采用动态改变优先数的办法，进程每运行一次优先数就减“1”。由于本实验是模拟处理器调度，所以，对被选中的进程并不实际的启动运行，而是执行:

优先数-1
要求运行时间-1

来模拟进程的一次运行。

提醒注意的是：在实际的系统中，当一个进程被选中运行时，必须恢复进程的现场，让它占有处理器运行，直到出现等待事件或运行结束。在这里省去了这些工作。

5、进程运行一次后，若要求运行时间 $\neq 0$ ，则再将它加入队列（按优先数大小插入，且置队首标志）；若要求运行时间 $=0$ ，则把它的状态修改成“结束”（E），且退出队列。

6、若“就绪”状态的进程队列不为空，则重复上面（4）和（5）的步骤，直到所有进程都成为“结束”状态。

7、在所设计的程序中应有显示或打印语句，能显示或打印每次被选中进程的进程名以及运行一次后进程队列的变化。

可以为五个进程任意确定一组“优先数”和“要求运行时间”，启动所设计的处理器调度程序，显示或打印逐次被选中进程的进程名以及进程控制块的动态变化过程。

（二）设计一个按时间片轮转法实现处理器调度的程序。

[提示]：

1、假定系统有五个进程，每一个进程用一个进程控制块 PCB 来代表。进程控制块的格式为：

进程名
指针
要求运行时间
已运行时间
状态

进程名——作为进程的标识，假设五个进程的进程名分别为 Q_1, Q_2, Q_3, Q_4, Q_5 。

指针——进程按顺序排成循环队列，用指针指出下一个进程的进程控制块的首地址，最后一个进程的指针指出第一个进程的进程控制块首地址。

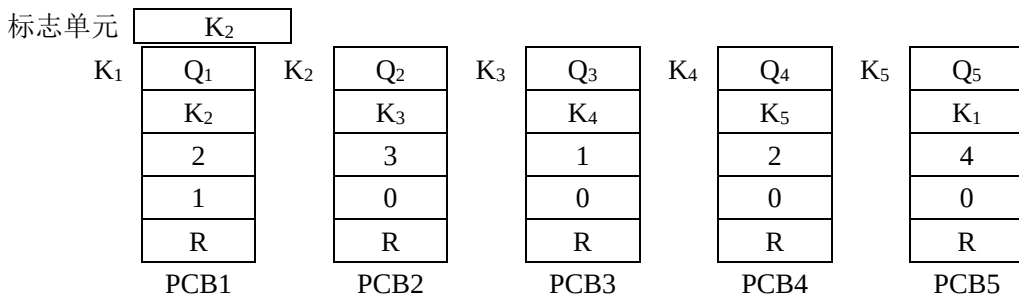
要求运行时间——假设进程需要运行的单位时间数。

已运行时间——假设进程已经运行的单位时间数，初始值为“0”。

状态——有两种状态，“就绪”和“结束”，初始状态都为“就绪”，用“R”表示。当一个进程运行结束后，它的状态为“结束”，用“E”表示。

2、每次运行所设计的处理器调度程序前，为每个进程任意确定它的“要求运行时间”。

3、把五个进程按顺序排成循环队列，用指针指出队列连接情况。另用一标志单元记录轮到运行的进程。例如，当前轮到 P_2 执行，则有：



4、处理器调度总是选择标志单元指示的进程运行。由于本实验是模拟处理器调度的功能，所以，对被选中的进程并不实际的启动运行，而是执行：

已运行时间+1

来模拟进程的一次运行，表示进程已经运行过一个单位的时间。

请同学注意：在实际的系统中，当一个进程被选中运行时，必须置上该进程可以运行的时间片值，以及恢复进程的现场，让它占有处理器运行，直到出现等待事件或运行满一个时间片。在这时省去了这些工作，仅用“已运行时间+1”来表示进程已经运行满一个时间片。

5、进程运行一次后，应把该进程的进程控制块中的指针值送到标志单元，以指示下一个轮到运行的进程。同时，应判断该进程的要求运行时间与已运行时间，若该进程的要求运行时间 $\neq$ 已运行时间，则表

示它尚未执行结束，应待到下一轮时再运行。若该进程的要求运行时间=已运行时间，则表示它已经执行结束，应指导它的状态修改成“结束”(E)且退出队列。此时，应把该进程的进程控制块中的指针值送到前面一个进程的指针位置。

6、若“就绪”状态的进程队列不为空，则重复上面的4和5的步骤，直到所有的进程都成为“结束”状态。

7、在所设计的程序中应有显示或打印语句，能显示或打印每次选中进程的进程名以及运行一次后进程队列的变化。

可以为五个进程任意确定一组“要求运行时间”，启动所设计的处理器调度程序，显示或打印逐次被选中的进程名以及进程控制块的动态变化过程。

四、实验内容

- 1、设计一个按背景材料中指定优先数的优先数调度算法实现处理器调度的程序
- 2、设计一个按时间片轮转法实现处理器调度的程序。

五、实验报告书写要求

- 1、程序中使用的数据结构及符号说明。

//程序中使用的数据结构示例

```
typedef struct _proc { /*进程结构体*/
    char name[32]; /*进程名*/
    int run_time; /*要求运行总时间*/
    int alloc_time; /*已经运行时间*/
    int req_time; /*还需要的运行时间*/
    int pri; /*优先级：时间片调度中未使用*/
    PROC_STATE state; /*进程状态*/
    struct _proc *next;
} PROC;
```

- 2、流程图。
- 3、打印一份源程序并附上注释。
- 4、打印程序运行时的初值和运行结果。要求如下：
 - i 进程控制块的初始状态。

--	--	--	--	--

- ii 选中运行的进程名以及选中进程运行后的各进程控制块状态。

--	--	--	--	--

对于 ii 要求每选中一个进程运行后都要打印。

实验 6 存储管理模拟

实验目的

存储管理的主要功能之一是合理地分配空间。请求页式管理是一种常用的虚拟存储管理技术。

本实验的目的是通过请求页式存储管理中页面置换算法模拟设计，了解虚拟存储技术的技术特点，掌握请求页式存储管理的页面置换算法。

实验要求

(1) 通过随机数产生一个指令序列，共 320 条指令。指令的地址按下述原则生成：

- ①50%的指令是顺序执行的；
- ②50%的指令是均匀分布在前地址部分；
- ③50%的指令是均匀分布在后地址部分。

具体的实施方法是：

- ①在 $[0, 319]$ 的指令之间随即选取一起点 m ；
- ②顺序执行一条指令，即执行地址为 $m+1$ 的指令；
- ③在前地址 $[0, m+1]$ 中随机选取一条指令并执行，该指令的地址为 m' ；
- ④顺序执行一条指令，其地址为 $m' + 1$ ；
- ⑤在后地址 $[m' + 2, 319]$ 中随机选取一条指令并执行；
- ⑥重复上述步骤①-⑤，直到执行 320 次指令。

(2) 将指令序列变换为页地址流

设：①页面大小为 1k；

②用户内存容量为 4 页到 32 页；

③用户虚存容量为 32k。

在用户虚存中，按每 k 存放 10 条指令排在虚存地址，即 320 条指令在虚存中的存放方式为：

第 0 条-第 9 条指令为第 0 页（对应虚存地址为 $[0, 9]$ ）；

第 10 条-第 19 条指令为第一页（对应虚存地址为 $[10, 19]$ ）；

... ..

第 310 条~第 319 条指令为第 31 页（对应虚地址为 $[310, 319]$ ）。

按以上方式，用户指令可组成 32 页。

(3) 计算并输出下述各种算法在不同内存容量下的命中率。

- ①先进先出的算法（FIFO）；
- ②最近最少使用算法（LRR）；
- ③最佳淘汰算法（OPT）先淘汰最不常用的页地址；
- ④最少访问页面算法（LFR）；
- ⑤最近最不经常使用算法（NUR）。

其中③和④为选择内容。

命中率=1-页面失效次数/页地址流长度

在本实验中，页地址流长度为 320，页面失效次数为每次访问相应指令时，该指令所对应的页不在内存的次数。

3、随机数产生办法，Linux 或 UNIX 系统提供函数 `strand()`和 `rand()`,分别进行初始化和产生随机数。例如：

`srand ()`;

语句可初始化一个随机数；

```
a[0]=10*rand()/65535*319+1;
```

```
a[1]=10*rand()/65535*a[0];
```

语句可用来产生 a[0]与 a[1]中的随机数。

实验内容

<任务>

设计一个虚拟存储区和内存工作区,并使用下列算法计算访问命中率.

进先出的算法 (FIFO)

最近最少使用的算法 (LRU)

最佳淘汰算法(OPT)

最少访问页面算法(LFU)

最近最不经常使用算法(NUR)

命中率=(1-页面失效次数)/页地址流长度

<程序设计>

本实验的程序设计基本上按照实验内容进行。即首先用 `srand()`和 `rand()`函数定义和产生指令序列,然后将指令序列变换成相应的页地址流,并针对不同的算法计算出相应的命中率。相关定义如下:

1 数据结构

(1)页面类型

```
typedef struct{
    int pn,pfn,counter,time;
}pl-type;
```

其中 pn 为页号,pfn 为面号, counter 为一个周期内访问该页面的次数, time 为访问时间.

(2) 页面控制结构

```
pfc_struct{
    int pn,pfn;
    struct pfc_struct *next;
}
typedef struct pfc_struct pfc_type;
pfc_type pfc_struct[total_vp],*freepf_head,*busypf_head;
pfc_type *busypf_tail;
```

其中 pfc[total_vp]定义用户进程虚页控制结构,

*freepf_head 为空页面头的指针,

*busypf_head 为忙页面头的指针,

*busypf_tail 为忙页面尾的指针.

2. 函数定义

(1) Void initialize():初始化函数,给每个相关的页面赋值.

(2) Void FIFO():计算使用 FIFO 算法时的命中率.

(3) Void LRU():计算使用 LRU 算法时的命中率.

(4) Void OPT():计算使用 OPT 算法时的命中率.

(5) Void LFU():计算使用 LFU 算法时的命中率.

(6) Void NUR():计算使用 NUR 算法时的命中率.

3.变量定义

(1)int a[total_instruction]: 指令流数据组.

(2)int page[total_instruction]: 每条指令所属的页号.

(3)int offset[total_instruction]: 每页装入 10 条指令后取模运算页号偏移值.

(4)int total_pf: 用户进程的内存页面数.

(5)int disaffect: 页面失效次数.

4.程序参考源码及结果

<程序>

```
#define TRUE 1
#define FALSE 0
#define INVALID -1
#define NULL 0

#define total_instruction 320 /*指令流长*/
#define total_vp 32 /*虚页长*/
#define clear_period 50 /*清 0 周期*/

typedef struct /*页面结构*/
{
    int pn; /*页号 logic number*/
    int pfn; /*页面框架号 physical frame number*/
    int counter; /*计数器*/
    int time; /*时间*/
}pl_type;

pl_type pl[total_vp]; /*页面线性结构---指令序列需要使用地址*/

typedef struct pfc_struct /*页面控制结构，调度算法的控制结构*/
{
    int pn;
    int pfn;
    struct pfc_struct *next;
}pfc_type;

pfc_type pfc[total_vp], *freepf_head, *busypf_head, *busypf_tail;

int diseffect, a[total_instruction]; /* a[]为指令序列*/

int page[total_instruction], offset[total_instruction];/*地址信息*/

int initialize(int);
int FIFO(int);
int LRU(int);
int LFU(int);
int NUR(int); /*not use recently*/
int OPT(int);

int main( )
{
    int s,i,j;
```

```

    srand(10*getpid());
    /*由于每次运行时进程号不同，故可用来作为初始化随机数
    队列的“种子”*/

```

```

s=(float)319*rand( )/32767/32767/2+1; /*正态分布*/

```

```

for(i=0;i<total_instruction;i+=4) /*产生指令队列*/

```

```

{

```

```

    if(s<0||s>319)

```

```

    {

```

```

        printf("When i==%d,Error,s==%d\n",i,s);

```

```

        exit(0);

```

```

    }

```

```

    a[i]=s; /*任选一指令访问点 m*/

```

```

    a[i+1]=a[i]+1; /*顺序执行一条指令*/

```

```

    a[i+2]=(float)a[i]*rand( )/32767/32767/2; /*执行前地址指令 m*/

```

```

    a[i+3]=a[i+2]+1; /*顺序执行一条指令*/

```

```

    s=(float)(318-a[i+2])*rand( )/32767/32767/2+a[i+2]+2;

```

```

    if((a[i+2]>318)||(s>319))

```

```

        printf("a[%d+2],a number which is :%d and s==%d\n",i,a[i+2],s);

```

```

    }

```

```

for (i=0;i<total_instruction;i++) /*将指令序列变换成页地址流*/

```

```

{

```

```

    page[i]=a[i]/10;

```

```

    offset[i]=a[i]%10;

```

```

}

```

```

for(i=4;i<=32;i++) /*用户内存工作区从 4 个页面到 32 个页面*/

```

```

{

```

```

    printf("---%2d page frames---\n",i);

```

```

    FIFO(i);

```

```

    LRU(i);

```

```

    LFU(i);

```

```

    NUR(i);

```

```

    OPT(i);

```

```

}

```

```

return 0;

```

```

}

```

```

/*初始化相关数据结构 total_pf 表示内存的块数 */

```

```

int initialize(int total_pf)

```

```

{

```

```

int i;
diseffect=0;
for(i=0;i<total_vp;i++)
{

    pl[i].pfn=INVALID;          /*置页面控制结构中的页号，页面为空*/
    pl[i].counter=0;            /*页面控制结构中的访问次数为 0*/
    pl[i].time=-1;              /*访问的时间*/
}

for(i=0;i<total_pf-1;i++)/*建立 pfc[i-1]和 pfc[i]之间的链接*/
{
    pfc[i].next=&pfc[i+1];
    pfc[i].pfn=i;
}

pfc[total_pf-1].next=NULL;
pfc[total_pf-1].pfn=total_pf-1;
freepf_head=&pfc[0];          /*空页面队列的头指针为 pfc[0]*/
return 0;
}

int FIFO(int total_pf)          /*先进先出算法 total_pf:用户进程的内存页面数*/
{
    int i,j;
    pfc_type *p;                /*中间变量*/
    initialize(total_pf);        /*初始化相关页面控制用数据结构*/
    busypf_head=busypf_tail=NULL; /*忙页面队列头，队列尾链接*/
    for(i=0;i<total_instruction;i++)
    {
        if(pl[page[i]].pfn==INVALID) /*页面失效*/
        {
            diseffect+=1;            /*失效次数*/
            if(freepf_head==NULL)    /*无空闲页面*/
            {
                p=busypf_head->next;
                pl[busypf_head->pn].pfn=INVALID;
                freepf_head=busypf_head; /*释放忙页面队列的第一个页面*/
                freepf_head->next=NULL; /*表明还是缺页*/
                busypf_head=p;
            }
            p=freepf_head->next;
            freepf_head->pn=page[i];
            pl[page[i]].pfn=freepf_head->pfn;
            freepf_head->next=NULL; /*使 busy 的尾为 null*/
            if(busypf_tail==NULL)

```

```

        {
            busypf_tail=busypf_head=freepf_head;
        }
        else
        {
            busypf_tail->next=freepf_head;
            busypf_tail=freepf_head;
        }
        freepf_head=p;
    }
}
printf("FIFO:%6.4f\n",1-(float)diseffect/320);
return 0;

```

```

int LRU (int total_pf)          /*最近最久未使用算法 least recently used*/
{
    int min,minj,i,j,present_time; /*minj 为最小值下标*/
    initialize(total_pf);
    present_time=0;
    for(i=0;i<total_instruction;i++)
    {
        if(pl[page[i]].pfn==INVALID)          /*页面失效*/
        {
            diseffect++;
            if(freepf_head==NULL)          /*无空闲页面*/
            {
                min=32767;          /*设置最大值*/
                for(j=0;j<total_vp;j++)          /*找出 time 的最小值*/
                {
                    if(min>pl[j].time&&pl[j].pfn!=INVALID)
                    {
                        min=pl[j].time;
                        minj=j;
                    }
                }
                freepf_head=&pfc[pl[minj].pfn];    //腾出一个单元
                pl[minj].pfn=INVALID;
                pl[minj].time=0;
                freepf_head->next=NULL;
            }
            pl[page[i]].pfn=freepf_head->pfn;    //有空闲页面,改为有效
            pl[page[i]].time=present_time;
            freepf_head=freepf_head->next;    //减少一个 free 页面
        }
    }
    else
    {

```

```

        pl[page[i]].time=present_time;          //命中则增加该单元的访问次数
        present_time++;
    }
}
printf("LRU:%6.4f\n",1-(float)diseffect/320);
return 0;
}

int NUR(int total_pf)                          /*最近未使用算法 Not Used recently count 表示*/
{
    int i,j,dp,cont_flag,old_dp;
    pfc_type *t;
    initialize(total_pf);
    dp=0;

    for(i=0;i<total_instruction;i++)
    {
        if (pl[page[i]].pfn==INVALID)          /*页面失效*/
        {
            diseffect++;
            if(freepf_head==NULL)              /*无空闲页面*/
            {
                cont_flag=TRUE;
                old_dp=dp;

                while(cont_flag)
                {
                    if(pl[dp].counter==0&&pl[dp].pfn!=INVALID)
                        cont_flag=FALSE;
                    else
                    {
                        dp++;
                        if(dp==total_vp)
                            dp=0;
                        if(dp==old_dp)
                            for(j=0;j<total_vp;j++)
                                pl[j].counter=0;
                    }
                }
            }
            freepf_head=&pfc[pl[dp].pfn];
            pl[dp].pfn=INVALID;
            freepf_head->next=NULL;
        }

        pl[page[i]].pfn=freepf_head->pfn;
    }
}

```

```

    freepf_head->pn=page[i];

    freepf_head=freepf_head->next;
}
else
    pl[page[i]].counter=1;
if(i%clear_period==0)
    for(j=0;j<total_vp;j++)
        pl[j].counter=0;
}
printf("NUR:%6.4f\n",1-(float)diseffect/320);
return 0;
}

int OPT(int total_pf)      /*最佳置换算法*/
{
    int i,j, max,maxpage,d,dist[total_vp];
    pfc_type *t;
    initialize(total_pf);
    for(i=0;i<total_instruction;i++)
    {
        if(pl[page[i]].pfn==INVALID)      /*页面失效*/
        {
            diseffect++;
            if(freepf_head==NULL)          /*无空闲页面*/
            {
                for(j=0;j<total_vp;j++)
                {
                    if(pl[j].pfn!=INVALID)
                        dist[j]=32767;
                    else
                        dist[j]=0;
                }
                for(j=0;j<total_vp;j++)
                {
                    if((pl[j].pfn!=INVALID)&&(dist[j]==32767))
                    {
                        dist[j]=j;
                    }
                }
                max=0;
                for(j=0;j<total_vp;j++)
                    if(max<dist[j])
                    {
                        max=dist[j];

```

```

        maxpage=j;
    }
    freepf_head=&pfc[pl[maxpage].pfn];
    freepf_head->next=NULL;
    pl[maxpage].pfn=INVALID;
}
pl[page[i]].pfn=freepf_head->pfn;
freepf_head=freepf_head->next;
}
}
printf("OPT:%6.4f\n",1-(float)diseffect/320);
return 0;
}
/*该算法时根据已知的预测未知的，least frequency Used 是最不经常使用置换法*/
int LFU(int total_pf)
{
    int i,j,min,minpage;
    pfc_type *t;
    initialize(total_pf);
    for(i=0;i<total_instruction;i++)
    {
        if(pl[page[i]].pfn==INVALID)        /*页面失效*/
        {
            diseffect++;
            if(freepf_head==NULL)          /*无空闲页面*/
            {
                min=32767;
                /*获取 counter 的使用用频率最小的内存*/
                for(j=0;j<total_vp;j++)
                {
                    if(min>pl[j].counter&&pl[j].pfn!=INVALID)
                    {
                        min=pl[j].counter;
                        minpage=j;
                    }
                }
                freepf_head=&pfc[pl[minpage].pfn];
                pl[minpage].pfn=INVALID;
                pl[minpage].counter=0;
                freepf_head->next=NULL;
            }
            pl[page[i]].pfn=freepf_head->pfn;    //有空闲页面,改为有效
            pl[page[i]].counter++;
            freepf_head=freepf_head->next;      //减少一个 free 页面
        }
    }
    else

```

```

    {
        pl[page[i]].counter;
        pl[page[i]].counter=pl[page[i]].counter+1;
    }
}
printf("LFU:%6.4f\n",1-(float)diseffect/320);
return 0;
}

```

<结果一:>

```

4 page framesFIFO:0.2562LRU:0.2531OPT:0.3031LFU:0.2812NUR:0.2812
5 page framesFIFO:0.2969LRU:0.2906OPT:0.3500LFU:0.3219NUR:0.3094
6 page framesFIFO:0.3375LRU:0.3281OPT:0.3844LFU:0.3375NUR:0.3344
7 page framesFIFO:0.3563LRU:0.3563OPT:0.4031LFU:0.3563NUR:0.3500
8 page framesFIFO:0.3937LRU:0.3750OPT:0.4531LFU:0.3937NUR:0.3719
9 page framesFIFO:0.4219LRU:0.4094OPT:0.4844LFU:0.4156NUR:0.4062
10 page framesFIFO:0.4375LRU:0.4313OPT:0.5062LFU:0.4313NUR:0.4250
11 page framesFIFO:0.4813LRU:0.4625OPT:0.5531LFU:0.4500NUR:0.4656
12 page framesFIFO:0.5406LRU:0.4875OPT:0.5687LFU:0.4938NUR:0.4875
13 page framesFIFO:0.5500LRU:0.5188OPT:0.5969LFU:0.5062NUR:0.5437
14 page framesFIFO:0.5594LRU:0.5531OPT:0.6344LFU:0.5281NUR:0.5469
15 page framesFIFO:0.5687LRU:0.5844OPT:0.6687LFU:0.5469NUR:0.5813
16 page framesFIFO:0.5781LRU:0.5938OPT:0.6813LFU:0.5719NUR:0.5969
17 page framesFIFO:0.5906LRU:0.6156OPT:0.6969LFU:0.6156NUR:0.6156
18 page framesFIFO:0.6156LRU:0.6312OPT:0.7156LFU:0.6344NUR:0.6531
19 page framesFIFO:0.6687LRU:0.6656OPT:0.7344LFU:0.6531NUR:0.6719
20 page framesFIFO:0.6875LRU:0.6969OPT:0.7500LFU:0.6719NUR:0.6906
21 page framesFIFO:0.6906LRU:0.7094OPT:0.7688LFU:0.6969NUR:0.7188
22 page framesFIFO:0.7125LRU:0.7219OPT:0.7969LFU:0.7156NUR:0.7344
23 page framesFIFO:0.7156LRU:0.7406OPT:0.8125LFU:0.7250NUR:0.7812
24 page framesFIFO:0.7281LRU:0.7625OPT:0.8187LFU:0.7406NUR:0.7719
25 page framesFIFO:0.7469LRU:0.7750OPT:0.8344LFU:0.7594NUR:0.8000
26 page framesFIFO:0.8125LRU:0.8000OPT:0.8500LFU:0.7812NUR:0.8063
27 page framesFIFO:0.8313LRU:0.8187OPT:0.8594LFU:0.8031NUR:0.8281
28 page framesFIFO:0.8438LRU:0.8375OPT:0.8688LFU:0.8344NUR:0.8469
29 page framesFIFO:0.8688LRU:0.8531OPT:0.8750LFU:0.8562NUR:0.8562
30 page framesFIFO:0.8781LRU:0.8719OPT:0.8781LFU:0.8750NUR:0.8688
31 page framesFIFO:0.8938LRU:0.8750OPT:0.8844LFU:0.8844NUR:0.8906
32 page framesFIFO:0.9000LRU:0.9000OPT:0.9000LFU:0.9000NUR:0.9000

```

<分析>

从上述结果可知，在内存页面数较少（4~5 页）时，五种算法的命中率差别不大，都是 30%左右。在内存页面为 7~18 个页面之间时，5 种算法的访内命中率大致在 35%~60%之间变化。但是，FIFO 算法与 OPT 算法之间的差别一般在 6~10 个百分点左右。在内存页面为 25~32 个页面时，由于用户进程的所有指令基本上都已装入内存，使命中率增加，从而算法之间的差别不大。

比较上述 5 种算法，以 OPT 算法的命中率最高，NUR 算法次之，再就是 LFU 算法和 LRU 算法，其次是 FIFO 算法。就本问题，在 15 页之前，FIFO 的命中率比 LRU 的高。

<结果二:>

4 page framesFIFO:0.2594LRU:0.2562OPT:0.2687LFU:0.2437NUR:0.2625
5 page framesFIFO:0.3000LRU:0.3000OPT:0.3000LFU:0.2969NUR:0.2875
6 page framesFIFO:0.3375LRU:0.3281OPT:0.3281LFU:0.3094NUR:0.3281
7 page framesFIFO:0.3563LRU:0.3563OPT:0.3688LFU:0.3312NUR:0.3469
8 page framesFIFO:0.4031LRU:0.4094OPT:0.3875LFU:0.3406NUR:0.3781
9 page framesFIFO:0.4156LRU:0.4281OPT:0.4156LFU:0.3656NUR:0.4125
10 page framesFIFO:0.4281LRU:0.4469OPT:0.4313LFU:0.3750NUR:0.4406
11 page framesFIFO:0.4531LRU:0.4688OPT:0.4594LFU:0.4281NUR:0.4656
12 page framesFIFO:0.4656LRU:0.4813OPT:0.4906LFU:0.4375NUR:0.4938
13 page framesFIFO:0.4750LRU:0.5000OPT:0.5219LFU:0.4625NUR:0.5312
14 page framesFIFO:0.4906LRU:0.5125OPT:0.5375LFU:0.4938NUR:0.5500
15 page framesFIFO:0.5312LRU:0.5250OPT:0.5625LFU:0.5281NUR:0.5563
16 page framesFIFO:0.5406LRU:0.5625OPT:0.5813LFU:0.5531NUR:0.5844
17 page framesFIFO:0.5906LRU:0.5813OPT:0.6188LFU:0.5750NUR:0.6031
18 page framesFIFO:0.6000LRU:0.5906OPT:0.6344LFU:0.5906NUR:0.6250
19 page framesFIFO:0.6312LRU:0.6156OPT:0.6438LFU:0.6219NUR:0.6438
20 page framesFIFO:0.6406LRU:0.6344OPT:0.6625LFU:0.6438NUR:0.6750
21 page framesFIFO:0.6969LRU:0.6594OPT:0.6875LFU:0.6656NUR:0.6937
22 page framesFIFO:0.7000LRU:0.6781OPT:0.7125LFU:0.6813NUR:0.6844
23 page framesFIFO:0.7156LRU:0.6906OPT:0.7312LFU:0.7188NUR:0.6969
24 page framesFIFO:0.7438LRU:0.7219OPT:0.7531LFU:0.7438NUR:0.7469
25 page framesFIFO:0.7594LRU:0.7562OPT:0.7656LFU:0.7656NUR:0.7719
26 page framesFIFO:0.7750LRU:0.7812OPT:0.7937LFU:0.7781NUR:0.7781
27 page framesFIFO:0.8125LRU:0.7969OPT:0.8094LFU:0.8125NUR:0.7969
28 page framesFIFO:0.8344LRU:0.8313OPT:0.8281LFU:0.8313NUR:0.8406
29 page framesFIFO:0.8406LRU:0.8594OPT:0.8531LFU:0.8375NUR:0.8406
30 page framesFIFO:0.8625LRU:0.8781OPT:0.8750LFU:0.8562NUR:0.8594
31 page framesFIFO:0.8812LRU:0.8812OPT:0.8906LFU:0.8781NUR:0.8656
32 page framesFIFO:0.9000LRU:0.9000OPT:0.9000LFU:0.9000NUR:0.9000

<分析>

从结果可以看出，FIFO 的命中率竟然比 OPT 的还高。至于为什么，请探讨。

实验 7 文件系统与 Shell

实验目的

通过一个简单多用户文件系统的设计，加深理解文件系统的内部功能及内部实现。

实验要求

为 linux 系统设计一个简单的二级文件系统。要求做到以下几点：

(1) 可以实现下列几条命令（至少 4 条）；

login	用户登陆
dir	列文件目录
create	创建文件
delete	删除文件
open	打开文件
close	关闭文件
read	读文件
write	写文件

(2) 列目录时要列出文件名、物理地址、保护码和文件长度；

(3) 源文件可以进行读写保护。

实验提示

(1) 首先应确定文件系统的数据结构：主目录、子目录及活动文件等。主目录和子目录都以文件的形式存放于磁盘，这样便于查找和修改。

(2) 用户创建的文件，可以编号存储于磁盘上。如 file0, file1, file2...并以编号作为物理地址，在目录中进行登记。

实验内容

<任务>

为 Linux 系统设计一个简单的二级文件系统。要求做到以下几点：

1. 可以实现下列几条命令：

login	用户登录
dir	列目录
create	创建文件
delete	删除文件
open	打开文件
close	关闭文件
read	读文件
write	写文件

2. 列目录时要列出文件名，物理地址，保护码和文件长度

3. 源文件可以进行读写保护

<程序设计>

(1) 设计思想

本文件系统采用两级目录，其中第一级对应于用户账号，第二级对应于用户帐号下的文件。另外，为了简便文件系统未考虑文件共享，文件系统安全以及管道文件与设备文件等特殊内容。对这些内容感兴趣的读

者，可以在本系统的程序基础上进行扩充。

(2)主要数据结构

I 节点

```
struct inode{
struct inode  *i_forw;
struct inode  *i_back;
char I_flag;
unsigned int i_into;      /*磁盘 i 节点标号*/
unsigned int i_count;     /*引用计数*/
unsigned short di_number; /*关联文件书，当为 0 时，则删除该文件*/
unsigned short di_mode;   /*存取权限*/
unsigned short di_uid;    /*磁盘 i 节点用户*/

unsigned short di_gid;    /*磁盘 i 节点组*/
```

```
Unsigned int di_addr[NADDR];      /*物理块号*/
```

b) 磁盘 i 结点

Struct dinode

```
{
    unsigned short di_number;      /*关联文件数*/
    unsigned short di_mode;        /*存取权限*/

    unsigned short di_uid;
    unsigned short di_gid;
    unsigned long di_size;          /*文件大小*/
    unsigned int di_addr[NADDR];   /*物理块号*/
```

c)目录项结构

Struct direct

```
{
    char d_name[DIRSIZ];           /*目录名*/
    unsigned int d_ino;             /*目录号*/
}
```

d)超级块

Struct filsys

```
{
    unsigned short s_ise;          /*i 节点块数*/
    unsigned long s_fsize;         /*数据块数*/

    unsigned int s_nfree;          /*空闲块数*/
    unsigned short s_pfree;        /*空闲块指针*/
    unsigned int s_free[NICFREE];  /*空闲块堆栈*/

    unsigned int s_ninode;         /*空闲 i 节点数*/
    unsigned short s_pinode;       /*空闲 i 节点指针*/
```

```

unsigned int s_inode[NICINOD]; /*空闲 i 节点数组*/
unsigned int s_rinode; /*铭记 i 节点*/
char s_fmod; /*超级块修改标志*/
};

```

e) 用户密码

```

Struct pwd
{
    unsigned short P_uid;
    unsigned short P_gid;
    char password[PWOSIZ];
}

```

f) 目录

```

Struct dir
{
    struct direct direct[DIRNUM];
    int size;
}

```

g). 查找 i 内存节点的 hash 表

```

Struct hinode
{
    struct inode *iforw;
}

```

h). 系统打开表

```

Struct file
{
    char f_flag; /*文件操作标志*/
    unsigned int f_count; /*引用计数*/
    struct inode *f_inode; /*指向内存节点*/
    unsigned long f_off; /*读/写指针*/
}

```

i) 用户打开表

```

Struct user
{
    unsigned short u_default_mode;
    unsigned short u_uid; /*用户标志*/
    unsigned short u_gid; /*用户组标志*/
    unsigned short u_ofile[NOFILE]; /*用户打开表*/
}

```

3. 主要函数

- (1) i 节点内容获取函数 `iget()` (详细描述略)。
- (2) 节点内容释放函数 `iput()` (详细描述略)。
- (3) 目录创建函数 `mkdir()` (详细描述略)。
- (4) 目录搜索函数 `namei()` (详细描述略)。
- (5) 磁盘块分配函数 `ballocc()` (详细描述略)。
- (6) 磁盘块释放函数 `bfree()` (详细描述略)。
- (7) 分配 i 节点区函数 `iallocc()` (详细描述略)。

- (8) 解释结点区函数 ifree () (详细描述略)。
- (9) 搜索当前目录下文件的函数 iname () (详细描述略)。
- (10) 访问控制函数 access () (详细描述略)。
- (11) 显示目录和文件函数_dir () (详细描述略)。
- (12) 改变当前目录用函数 chdir () (详细描述略)。
- (13) 打开文件函数 open () (详细描述略)。
- (14) 创建文件函数 create () (详细描述略)。
- (15) 读文件用函数 read () (详细描述略)。
- (16) 读文件用函数 write () (详细描述略)。
- (17) 用户登陆函数 login () (详细描述略)。
- (18) 用户退出函数 logout () (详细描述略)。
- (19) 文件系统格式化函数 format () (详细描述略)。
- (20) 进入文件系统函数 install () (详细描述略)。
- (21) 关闭文件函数 close () (详细描述略)。
- (22) 退出文件系统函数 halt () (详细描述略)。
- (23) 文件删除函数 delecte () (详细描述略)。

4. 主程序说明

begin

- Step1 对磁盘进行格式化
- Step2 调用 install (), 进入文件系统
- Step3 调用_dir (), 显示当前目录
- Step4 调用 login (), 用户注册
- Step5 调用 mkdir () 和 chdir () 创建目录
- Step6 调用 create (), 创建文件 0
- Step7 分配缓冲区
- Step8 写文件 0
- Step9 关闭文件 0 和释放缓冲
- Step10 调用 mkdir () 和 chdir () 创建子目录
- Step11 调用 create (), 创建文件 1
- Step12 分配缓冲区
- Step13 写文件 1
- Step14 关闭文件 1 和释放缓冲
- Step15 调用 chdir 将当前目录移到上一级
- Step16 调用 create (), 创建文件 2
- Step17 分配缓冲区
- Step18 调用 write (), 写文件 2
- Step19 关闭文件 1 和释放缓冲
- Step20 调用 delecte (), 删除文件 0
- Step21 调用 create (), 创建文件 1
- Step22 为文件 3 分配缓冲区
- Step23 调用 write (), 写文件 2
- Step24 关闭文件 3 并释放缓冲区
- Step25 调用 open(), 打开文件 2
- Step26 为文件 2 分配缓冲
- Step27 写文件 3 后关闭文件 3
- Step28 释放缓冲

Step29 用户退出 (logout)

Step30 关闭 (halt)

End

由上述的描述过程可知，该文件系统实际是为用户提供一个解释执行相关命令的环境。主程序中的大部分语句都被用来执行相应的命令。

下面我们给出每个过程的相关 C 语言程序。读者也可以使用这些子过程，编写一个用 Shell 控制的文件系统界面。

<程序>

1.编写管理文件 makefile

本文件系统程序用编写 makefile 管理工具进行管理。其内容如下：

```
*****/
```

```
/******
```

```
makefile
```

```
*****/
```

```
filsys:main.o iallfre.o ballfre.o name.o access.o log.o close.o creat.o delete.o dir.o
```

```
open.o rdwt.o format.o install.o halt.o cc-o filsys main.o iallfre.o ballfre.o
```

```
name.o access.o log.o close.o creat.o delete.o dir.o open.o format.o install.o halt.o
```

```
main.o:main.c filesys.h
```

```
cc-c main.c
```

```
igetput.o: igetput.c filesys.h
```

```
cc-c igetput.c
```

```
iallfre.o: iallfre.c filesys.h
```

```
cc-c iallfre.c
```

```
ballfre.o: ballfre.c filesys.h
```

```
cc-c ballfre.c
```

```
name.o:name.c filesys.h
```

```
cc-c name.c
```

```
access.o:access.c filesys.h
```

```
cc-c access.c
```

```
log.o:log.c filesys.h
```

```
cc-c log.c
```

```
close.o:close.c filesys.h
```

```
cc-c close.c
```

```
creat.c:creat.c filesys.h
```

```
cc-c creat.c
```

```
delete.o:delete.c filesys.h
```

```
cc-c delete.c
```

```
dir.o:dir.c filesys.h
```

```
cc-c dir.c
```

```
open.o:open.c filesys.h
```

```
cc-c open.c
```

```
rdwt.o:rdwt.c filesys.h
```

```
cc-c rdwt.c
```

```
format.o:format.c filesys.h
```

```
cc-c format.c
install.o: install.c filesys.h
cc-c install.c
halt.o:halt.c
cc-c halt.c
```

附录 Linux 系统调用函数说明、参数值及定义

1、fork()

创建一个新进程

int fork()

其中返回 int 取值意义如下：

0：创建子进程，从子进程返回的 id 值

大于 0：从父进程返回的子进程 id 值

-1：创建失败

2、lockf(files, function, size)：

用作锁定文件的某些段或者整个文件，本函数适用的头文件为：

#include<unistd.h>

参数定义：

int lockf(files,function,size)

int files,function;

long size;

其中：files 是文件描述符；function 是锁定和解锁；1 表示锁定，0 表示解锁。size 是锁定和解锁的字节数，若用 0，表示从文件的当前位置到文件尾。

3、msgget(key, flag)：

获得一个消息的描述符，该描述符指定一个消息队列以便用于其他系统调用。

该函数使用头文件如下：

#include<sys/types.h>

#include<sys/ipc.h>

#include<sys/msg.h>

参数定义

int msgget(key,flag)

key_tkey;

int flag;

语法格式：msgqid=msgget(key,flag)

其中：msgid 是该系统调用返回的描述符，失败则返回-1；flag 本身由操作允许权和控制命令值相“或”得到。

如：IP_CREAT|0400 是否该队列应被创建；

IP_EXCL |0400 是否该队列的创建应是互斥的；等。

4、msgsnd(id, msgp, size, flag)：

发送一消息。

该函数是用头文件如下：

#include<sys/types.h>

#include<sys/ipc.h>

#include<sys/msg.h>

参数定义

```
int msgsnd(id,msgp,size,flag)
```

```
int id,size,flag;
```

```
struct msgbuf * msgp;
```

其中：id 是返回消息队列的描述符；msgp 是指向用户存储区的一个构造体指针，size 指示由 msgp 指向的数据结构中字符数组的长度；即消息的长度。这个数组的最大值由 MSG-MAX 系统可调用参数来确定。flag 规定当核心用尽内部缓冲空间时应执行的动作；若在标志 flag 中未设置 IPC_NOWAIT 位，则当该消息队列中字节数超过一最大值时，或系统范围的消息数超过某一最大值时，调用 msgsnd 进程睡眠。若是设置 IPC_NOWAIT，则在此情况下，msgsnd 立即返回。

5、msgrcv(id, msgp, size, type, flag) :

接受一消息。

该函数调用使用头文件如下：

```
#include<sy/types.h>
```

```
#include<sy/ipc.h>
```

```
#include<sy/msg.h>
```

参数定义

```
int msgrcv(id,msgp,size,type,flag)
```

```
int id,size,type,flag;
```

```
struct msgbuf * msgq;
```

```
struct sgbuf{long mtpc;char mtext[]};
```

语法格式：

```
count=msgrcv(id,msgp,size,type,flag)
```

其中：id 是用来存放欲接收消息的拥护数据结构的地址；size 是 msgp 中数据数组的大小；type 是用户要读的消息类型：

type 为 0:接收该队列的第一个消息；

type 为正：接收类型 type 的第一个消息；

type 为负：接收小于或等于 type 绝对值的最低类型的第一个消息。

flag 规定倘若该队列无消息，核心应当做什么事，如果此时设置了 IPC_NOWAIT 标志，则立即返回，若在 flag 中设置了 MSG_NOERROR，且所接收的消息大小大于 size，核心截断所接受的消息。

count 是返回消息正文的字节数。

6、msgctl(id, cmd, buf) :

查询一个消息描述符的状态，设置它的状态及删除一个消息描述符。

调用该函数使用头文件如下：

```
#include<sy/types.h>
```

```
#include<sy/ipc.h>
```

```
#include<sy/msg.h>
```

参数定义

```
int msgctl(id,cmd,buf)
```

```
int id,cmd;
```

```
struct msgbuf * msgq;
```

```
struct msqid_ds * buf;
```

其中：函数调用成功时返回 0，调用不成功时返回-1。id 用来识别该消息的描述符；cmd 规定命令的类型。

IPC_START 将与 id 相关联的消息队列首标读入 buf。

IPC_SET 为这个消息序列设置有效的用户和小组标识及操作允许权和字节的数量。

IPC_RMID 删除 id 的消息队列。

buf 是含有控制参数或查询结果的用户数据结构的地址。

附：msgid_ds 结构定义如下：

```
struct msgid_ds
{struct ipc_perm msg_perm;    /*许可权结构*/
shot padl[7];                /*由系统使用*/
ushort onsg_qnum;            /*队列上消息数*/
ushort msg_qbytes;          /*队列上最大字节数*/
ushort msg_lspid;            /*最后发送消息的 PID*/
ushort msg_lrpid;            /*最后接收消息的 PID*/
time_t msg__stime;          /*最后发送消息的时间*/
time_t msg_rtime;            /*最后接收消息的时间*/
me_t msg_ctime;              /*最后更改时间*/
};

struct ipc_perm
{ushort uid;                 /*当前用户 id*/
ushort gid;                   /*当前进程组 id*/
ushort cuid;                  /*创建用户 id*/
ushort cgid                    /*创建进程组 id*/
ushort mode;                  /*存取许可权*/
{shot patl;long pad2}        /*由系统使用*/
};
```

7、shmget (key, size, flag) :

获得一个共享存储区。

该函数使用头文件如下：

```
#include<sy/types.h>
```

```
#include<sy/ipc.h>
```

```
#include<sy/shm.h>
```

语法格式：

```
shmid=shmaget(key,size,flag)
```

参数定义：

```
int shmaget(key,size,flag)
```

```
key_t key;
```

```
int size,flag;
```

其中：size 是存储区的字节数，key 和 flag 与系统调用 msgget 中的参数含义相同。

附：

操作允许权	八进制数
用户可读	00400
用户可读	00200
小组可读	00040
小组可读	00020
其他可读	00004
其他可读	00002

控制命令	值
IPC_CREAT	0001000
IPC_EXCL	0002000

如:shmids=shmget(key,size,(IPC_CREAT|0400)) ;
创建一个关键字为 key,长度为 size 的共享存储区。

8、shmat(id, addr, flag) :

从逻辑上将一个共享存储区附接到进程的虚拟地址空间上。

该函数调用使用头文件如下:

```
#include<sys/types.h>
#include<sys/ipc.h>
#include<sys/msg.h>
```

参数定义:

```
char * shmat(id,addr,flag)
int id,flag;
char * addr;
```

语法格式: virtaddr=shmat(id,addr,flag)

其中: id 是共享存储区的标识符, addr 是用户要使用共享存储区附接的虚地址, 若 addr 是 0, 系统是否对应用户规定的地址做舍入操作。如果 flag 中设置了 shm_rnd 即表示操作系统在必要时舍去这个地址。如果设置了 shm_rndonly, 即表示只允许读操作。viraddr 是附接的虚地址。

9、shmdt(addr) :

把一个共享存储区从指定进程的虚地址空间分开。

调用该函数使用头文件:

```
#include<sys/types.h>
#include<sys/ipc.h>
#include<sys/mshm.h>
```

参数定义:

```
int shmdt(addr)
char * addr
```

其中, 当调用成功时, 返回 0 值, 调用不成功, 返回-1, addr 是系统调用 shmat 所返回的地址。

10、shmctl(id, cmd, buf) :

对与共享存储区关联的各种参数进行操作, 从而对共享存储区进行控制。

调用该函数使用头文件:

```
#include<sys/types.h>
#include<sys/ipc.h>
#include<sys/shm.h>
```

参数定义:

```
int shmctl(id,cmd,buf)
int id,cmd;
struct shmids * buf;
```

其中: 调用成功返回 0, 否则返回-1。id 为被共享存储区的标识符。cmd 规定操作的类型。规定如下: IPC_STAT:返回包含在指定的 shmids 相关数据结构中的状态信息, 并且把它放置在用户存储区中的*buf 指针所指的数据结构中。执行此命令的进程必须有读取允许权。

IPC_SET: 对于指定的 shmid, 为它设置有效用户和小组标识和操作存取权。

IPC_RMID: 删除指定的 shmid 以及与它相关的共享存储区的数据结构。

SHM_LOCK: 在内存中锁定指定的共享存储区, 必须是超级用户才可以进行此项操作。

Buf 是一个用户级数据结构地址。

附:

```
shmid_ds
{struct ipc_perm shm_perm;      /*允许权结构*/
int shm_segsz;                  /*段大小*/
int padl;                       /*由系统使用; */
ushort shm_lpid;                /*最后操作的进程 id; */
ushort shm_cpid;                /*创建者的进程 id; */
ushort shm_nattch;              /*当前附界数; */
short pad2;                     /*由系统使用; */
time_t shm_atime;               /*最后附接时间*/
time_t shm_dtime;               /*最后段接时间*/
time_t shm_ctime;               /*最后修改时间*/
}
```

11、signal(sig, function):

允许调用进程控制软中断信号的处理。

头文件为:

```
#include<signal.h>
```

参数定义:

```
signal(sig,function);
```

```
int sig;
```

```
void(*func)();
```

其中: sig 的值是:

SIGHVP 挂起

SIGINT 键盘按^c 键或 break 键

SIGQUIT 键盘按 quit 键

SIGILL 非法指令

SIGIOT IOT 指令

SIGEMT EMT 指令

SIGFPE 浮点运算溢出

SIGKILL 要求终止进程

SIGBUS 总线错

SIGSEGV 段违例

SIGSYS 系统调用参数错

SIGPIPE 向无读者管道上写

SIGALRM 闹钟

SIGTERM 软件终结

SIGUSR1 用户定义信号

SIGUSR2 第二个用户定义信号

SIGCLD 子进程死

SIGPWR 电源故障

function 的解释如下:

SIG_DEL: 缺省操作。对除 SIGPWR 和 SIGCLD 外所有信号的缺省操作是进程终结对信号 SIGQUIT,SIGILL,SIGTRA,SIGIOT,SIGEMT,SIGFPE,SIGBUS,SIGSEGV 和 SIGSYS 它产生一内存映像文件。

SIG_IGN: 忽视该信号的出现。

Function:在该进程中的一个函数地址，在核心返回用户态时，它以软中断信号的序号作为参数调用该函数，对除了信号 SIGILL,SIGTRAP 和 SIGTWR 以外的信号，核心自动地重新设置软中断信号处理程序的值为 SIG_DEL，一个进程不能捕获 SIGKILL 信号。